

CSE145 - Introduction to Computing

Assignment 2: Turing Machines - Design, Analysis and Simulation

Name	Muhammad Taahir
ERP No.	35618
GitHub repository	https://github.com/MTaahir-00/35618-cse145-a2-tm

Personalization block

Value	Result	Working
R	18	last two digits of 35618
V	2	$18 \bmod 4 = 2$, because $18 = 4 \times 4 + 2$
B	10010	$18 = 16 + 2$, so 18 in binary is 10010
n	6	digit sum $3+5+6+1+8 = 23$, $23 \bmod 5 = 3$, and $3 + 3 = 6$

My variant (V = 2)

Question	Task
Q1	Divisible-by-M recognizer, $M = 5$
Q2	Scaled-count recognizer: $a^{2n}b^n$ (my main test string uses $n = 6$)
Q3	Binary transformer: $x2$ (append a 0)
Q4	Palindrome family: even-length palindromes over $\{a,b\}$
Q5	Marked comparison: $\{ a^i \# b^j : i \geq j \}$
Q6	Debug the broken machine (same for everyone)

Notes on how I wrote this up

All the machines are single-tape and were built and run in AutomataVerse. The simulator shows the blank as a capital B, so I use B for blank in this document. In Q2 and Q4 the small b is a normal input letter, not the blank. The state names in my tables (q0, q1, ...) match the ones in my screenshots.

For Q1 to Q5 I didn't draw a reject state. When the machine has no transition for the symbol it is reading, AutomataVerse stops and shows Rejected, so I treat that as my reject state. The tables show which cases end this way. In Q6 the reject state q3 is drawn.

Each run has its screenshot and the JSON code of the machine, exactly as exported from AutomataVerse, right below it. The same JSON files are also attached to this PDF, and they are in the GitHub repo with the screenshots (the share link for each run is on the matching line of README.md).

Q1 - Divisible by 5 ($V = 2$, $M = 5$, $B = 10010$)

Task A - Strategy

I read the bits left to right, biggest bit first. The machine only has to remember the remainder (mod 5) of what it has read so far, so I made 5 states, q_0 to q_4 , one for each remainder. q_0 is the start state because nothing read yet is 0, which also means the empty string and 0 get accepted. Reading a bit b in the state with remainder r takes it to remainder $(2r + b) \bmod 5$, because adding a bit on the right doubles the number and adds b . Each move writes the same bit back and goes right. When it hits the blank, q_0 goes to the accept state q_5 , and q_1 to q_4 have no blank transition so the machine just stops and rejects.

Task B - Transition table

Current state	Read	Write	Move	Next state
q_0	0	0	R	q_0
q_0	1	1	R	q_1
q_0	B	B	R	q_5
q_1	0	0	R	q_2
q_1	1	1	R	q_3
q_1	any other: B	same	R	no transition: halts and rejects
q_2	0	0	R	q_4
q_2	1	1	R	q_0
q_2	any other: B	same	R	no transition: halts and rejects
q_3	0	0	R	q_1
q_3	1	1	R	q_2
q_3	any other: B	same	R	no transition: halts and rejects
q_4	0	0	R	q_3
q_4	1	1	R	q_4
q_4	any other: B	same	R	no transition: halts and rejects
q_5	-	-	-	accept state (halts, accepted)

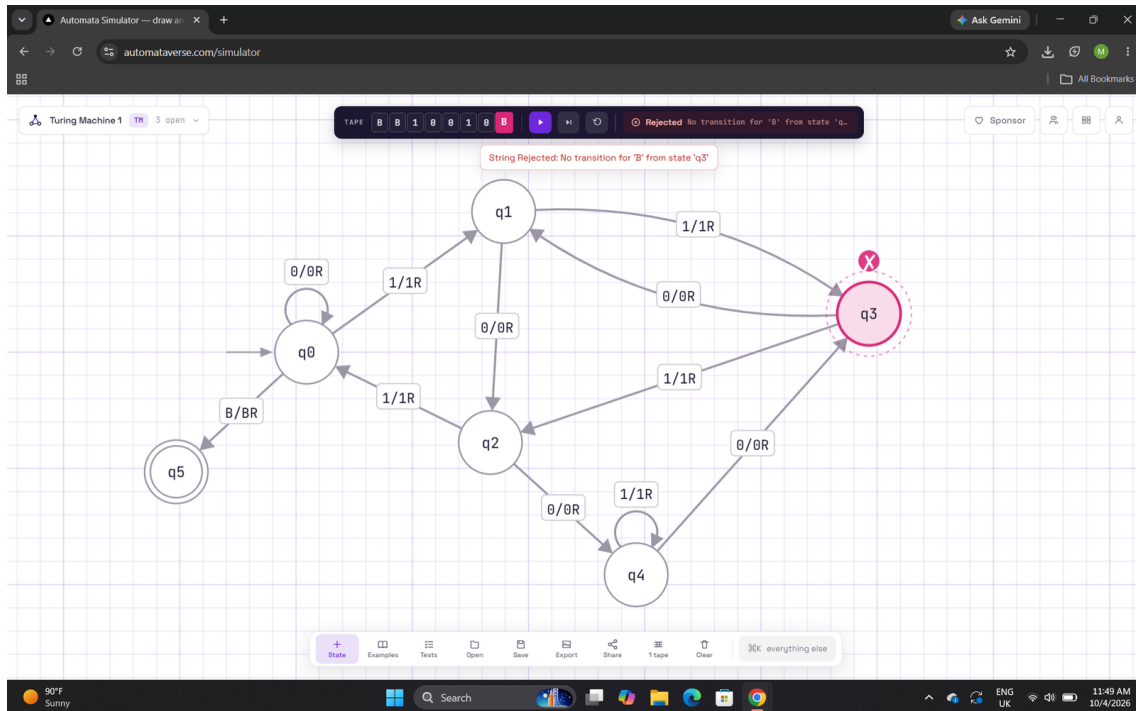
The number in each state name is the remainder so far (q_3 means remainder 3). Example: in q_3 reading 1 gives $(2 \times 3 + 1) \bmod 5 = 2$, so $(q_3, 1)$ goes to q_2 . The blank is only accepted from q_0 , so reading B in q_1 , q_2 , q_3 or q_4 is covered by the "no transition" rows.

Task C - Simulator runs

Trace of run 1: $q_0 \xrightarrow{-1} q_1 \xrightarrow{-0} q_2 \xrightarrow{-0} q_4 \xrightarrow{-1} q_4 \xrightarrow{-0} q_3$, then blank in q_3 has no transition, so it rejects. $18 \bmod 5 = 3$, so rejecting is right.

Run 1: my B = 10010 (decimal 18)

Result: **Rejected** (stopped in q3, remainder 3). This is the correct answer, because 18 is not divisible by 5.



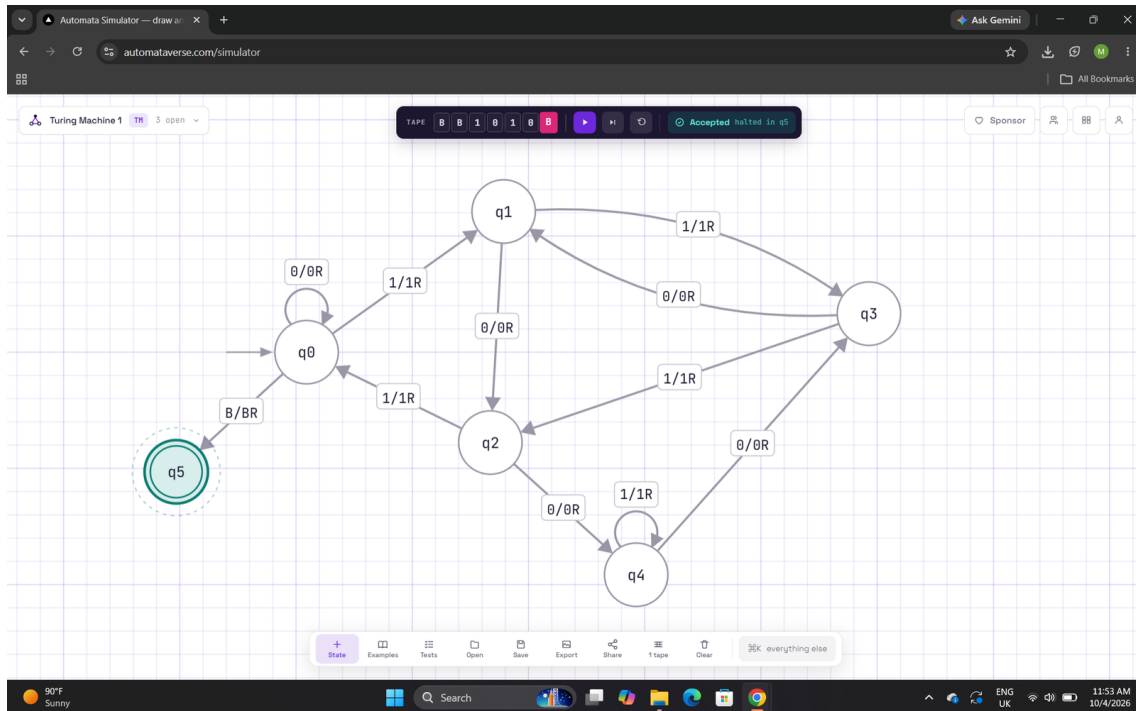
Repo files: q1_run1_B.json, q1_run1_B.png, README line "Q1 run 1"

JSON export (q1_run1_B.json):

```
{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 120,
      "y": 110
    },
    "q1": {
      "id": "q1",
      "x": 306.4950610051862,
      "y": -23.18768180292902
    },
    "q2": {
      "id": "q2",
      "x": 293.9095871049413,
      "y": 195.52051147345972
    },
    "q3": {
      "id": "q3",
      "x": 652.1466588560097,
      "y": 73.54863445393522
    },
    "q4": {
      "id": "q4",
      "x": 431.93957053843906,
      "y": 320.65729233984143
    },
    "q5": {
      "id": "q5",
      "x": -3.3415461363645704,
      "y": 223.2199882840145
    }
  },
  "transitionMap": {
    "q0": [
      {
        "label": "0/0R",
        "sourceid": "q0",
        "targetid": "q0"
      },
      {
        "label": "1/1R",
        "sourceid": "q0",
        "targetid": "q1"
      },
      {
        "label": "B/BR",
        "sourceid": "q0",
        "targetid": "q5"
      }
    ],
    "q1": [
      {
        "label": "0/0R",
        "sourceid": "q1",
        "targetid": "q2"
      },
      {
        "label": "1/1R",
        "sourceid": "q1",
        "targetid": "q3"
      }
    ],
    "q2": [
      {
        "label": "0/0R",
        "sourceid": "q2",
        "targetid": "q4"
      },
      {
        "label": "1/1R",
        "sourceid": "q2",
        "targetid": "q0"
      }
    ],
    "q3": [
      {
        "label": "0/0R",
        "sourceid": "q3",
        "targetid": "q1"
      },
      {
        "label": "1/1R",
        "sourceid": "q3",
        "targetid": "q2"
      }
    ],
    "q4": [
      {
        "label": "0/0R",
        "sourceid": "q4",
        "targetid": "q3"
      },
      {
        "label": "1/1R",
        "sourceid": "q4",
        "targetid": "q3"
      }
    ]
  },
  "finalNodes": [
    "q5"
  ],
  "nodeName": 6,
  "automataType": "TM",
  "inputString": "10010",
  "stageProps": {
    "x": 233.25806879985748,
    "y": 191.23985954749318,
    "scale": 1.4265583530550014,
    "draggable": true,
    "stateOutput": {},
    "tapeMode": "single",
    "inputString2": ""
  }
}
```

Run 2: 1010 (decimal 10, divisible by 5)

Result: **Accepted** (halted in q5). This is the correct answer, because $10 = 5 \times 2$.



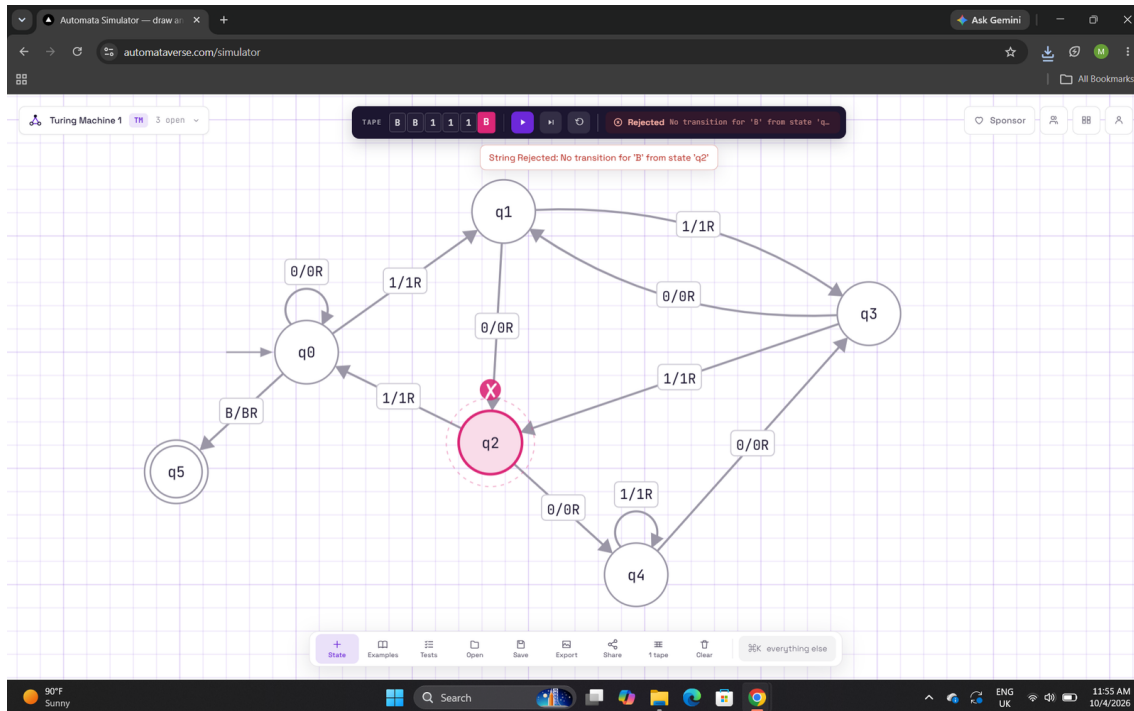
Repo files: q1_run2_div.json, q1_run2_div.png, README line "Q1 run 2"

JSON export (q1_run2_div.json):

```
{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 120,
      "y": 110
    },
    "q1": {
      "id": "q1",
      "x": 306.4950610051862,
      "y": -23.18768180292902
    },
    "q2": {
      "id": "q2",
      "x": 293.9095871049413,
      "y": 195.52051147345972
    },
    "q3": {
      "id": "q3",
      "x": 652.1466588560097,
      "y": 73.54863445393522
    },
    "q4": {
      "id": "q4",
      "x": 431.93957053843906,
      "y": 320.65729233984143
    },
    "q5": {
      "id": "q5",
      "x": -3.3415461363645704,
      "y": 223.2199882840145
    }
  },
  "transitionMap": {
    "q0": [
      {
        "label": "0/0R",
        "sourceid": "q0",
        "targetid": "q0"
      },
      {
        "label": "1/1R",
        "sourceid": "q0",
        "targetid": "q1"
      },
      {
        "label": "1/1R",
        "sourceid": "q0",
        "targetid": "q2"
      },
      {
        "label": "B/BR",
        "sourceid": "q0",
        "targetid": "q5"
      }
    ],
    "q1": [
      {
        "label": "0/0R",
        "sourceid": "q1",
        "targetid": "q2"
      },
      {
        "label": "1/1R",
        "sourceid": "q1",
        "targetid": "q3"
      }
    ],
    "q2": [
      {
        "label": "0/0R",
        "sourceid": "q2",
        "targetid": "q4"
      },
      {
        "label": "1/1R",
        "sourceid": "q2",
        "targetid": "q0"
      }
    ],
    "q3": [
      {
        "label": "0/0R",
        "sourceid": "q3",
        "targetid": "q1"
      },
      {
        "label": "1/1R",
        "sourceid": "q3",
        "targetid": "q2"
      }
    ],
    "q4": [
      {
        "label": "0/0R",
        "sourceid": "q4",
        "targetid": "q3"
      },
      {
        "label": "1/1R",
        "sourceid": "q4",
        "targetid": "q3"
      }
    ]
  },
  "finalNodes": [
    "q5"
  ],
  "nodeName": 6,
  "automataType": "TM",
  "inputString": "1010",
  "stageProps": {
    "x": 233.25806879985748,
    "y": 191.23985954749318,
    "scale": 1.4265583530550014,
    "draggable": true
  },
  "stateOutput": {},
  "tapeMode": "single",
  "inputString2": ""
}
```

Run 3: 111 (decimal 7, not divisible by 5)

Result: **Rejected** (stopped in q2, remainder 2). This is the correct answer, because $7 \bmod 5 = 2$.



Repo files: q1_run3_notdiv.json, q1_run3_notdiv.png, README line "Q1 run 3"

JSON export (q1_run3_notdiv.json):

```
{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 120,
      "y": 110,
      "x2": 306.4950610051862,
      "y2": -23.18768180292902,
      "id": "q1",
      "x": 306.4950610051862,
      "y": -23.18768180292902,
      "id": "q2",
      "x": 293.9095871049413,
      "y": 195.52051147345972,
      "id": "q3",
      "x": 652.1466588560097,
      "y": 73.54863445393522,
      "id": "q4",
      "x": 431.93957053843906,
      "y": 320.65729233984143,
      "id": "q5",
      "x": -3.3415461363645704,
      "y": 223.2199882840145
    },
    "transitionMap": {
      "q0": [
        {
          "label": "0/0R",
          "sourceid": "q0",
          "targetid": "q0"
        },
        {
          "label": "1/1R",
          "sourceid": "q0",
          "targetid": "q1"
        },
        {
          "label": "B/BR",
          "sourceid": "q0",
          "targetid": "q5"
        },
        {
          "label": "0/0R",
          "sourceid": "q1",
          "targetid": "q2"
        },
        {
          "label": "1/1R",
          "sourceid": "q1",
          "targetid": "q3"
        },
        {
          "label": "0/0R",
          "sourceid": "q2",
          "targetid": "q4"
        },
        {
          "label": "1/1R",
          "sourceid": "q2",
          "targetid": "q0"
        },
        {
          "label": "0/0R",
          "sourceid": "q3",
          "targetid": "q1"
        },
        {
          "label": "1/1R",
          "sourceid": "q3",
          "targetid": "q4"
        },
        {
          "label": "0/0R",
          "sourceid": "q4",
          "targetid": "q3"
        },
        {
          "label": "1/1R",
          "sourceid": "q4",
          "targetid": "q4"
        }
      ],
      "finalNodes": [
        "q5"
      ],
      "nodeName": 6,
      "automataType": "TM",
      "inputString": "111",
      "stageProps": {
        "x": 233.25806879985748,
        "y": 191.23985954749318,
        "scale": 1.4265583530550014,
        "draggable": true,
        "stateOutput": {},
        "tapeMode": "single",
        "inputString2": ""
      }
    }
  }
}
```

Task D - Why this machine only works for $M = 5$

All the arrows come from the formula $(2r + b) \bmod 5$, so the table only works for 5. With another M I'd need M states and every arrow would have to be worked out again using $\bmod M$. The accepting check would also point to the new remainder-0 state. Only the scanning to the right stays the same.

Q2 - $a^{2n}b^n$ ($V = 2, n = 6$)

Task A - Strategy

The language is a's followed by b's, with twice as many a's as b's. My machine crosses off two a's and one b each round. q0 turns the first a into X and q1 turns the next a into X. Then q3 goes right to the first b that isn't crossed off and makes it a Y, and q4 goes back left to the last X and steps right into q0 for the next round. If q0 finds a Y instead of an a, the a's are finished, so q2 walks over the Y's and accepts only if the next cell is blank (no b's left). If q1 can't find a second a or q3 can't find a b, the counts don't match and the machine rejects. The empty string is accepted in q0 right at the start ($k = 0$).

Task B - Transition table

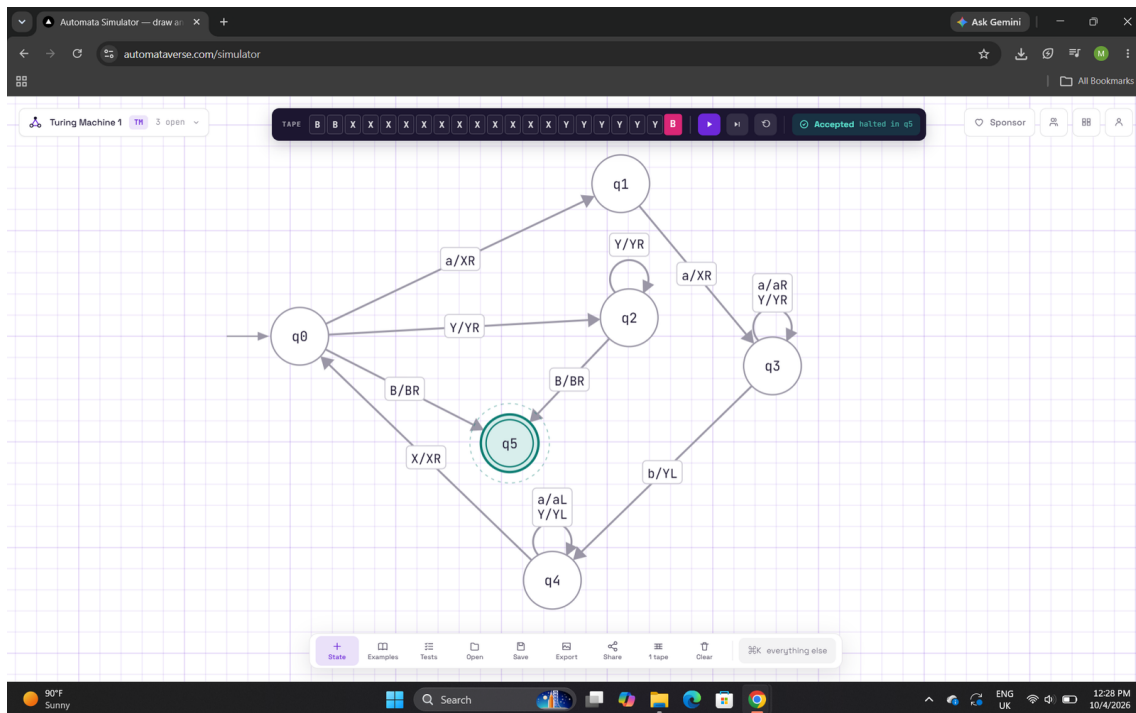
Current state	Read	Write	Move	Next state
q0	a	X	R	q1
q0	Y	Y	R	q2
q0	B	B	R	q5
q0	any other: X b	same	R	no transition: halts and rejects
q1	a	X	R	q3
q1	any other: B X Y b	same	R	no transition: halts and rejects
q2	Y	Y	R	q2
q2	B	B	R	q5
q2	any other: X a b	same	R	no transition: halts and rejects
q3	a	a	R	q3
q3	Y	Y	R	q3
q3	b	Y	L	q4
q3	any other: B X	same	R	no transition: halts and rejects
q4	a	a	L	q4
q4	Y	Y	L	q4
q4	X	X	R	q0
q4	any other: B b	same	R	no transition: halts and rejects
q5	-	-	-	accept state (halts, accepted)

What each state does: q0 = start of a round, marks the first a. q1 = marks the second a. q2 = final check that only Y's are left. q3 = go right to the first unmarked b. q4 = go back left to the last X. q5 = accept.

Task C - Simulator runs

Run 1: 12 a's then 6 b's (n = 6, so 2n = 12 a's and n = 6 b's)

Input: aaaaaaaaaabbbbbbb. Result: **Accepted** (halted in q5). Correct, the string is in the language. The tape ends as 12 X and 6 Y, so every symbol got matched.



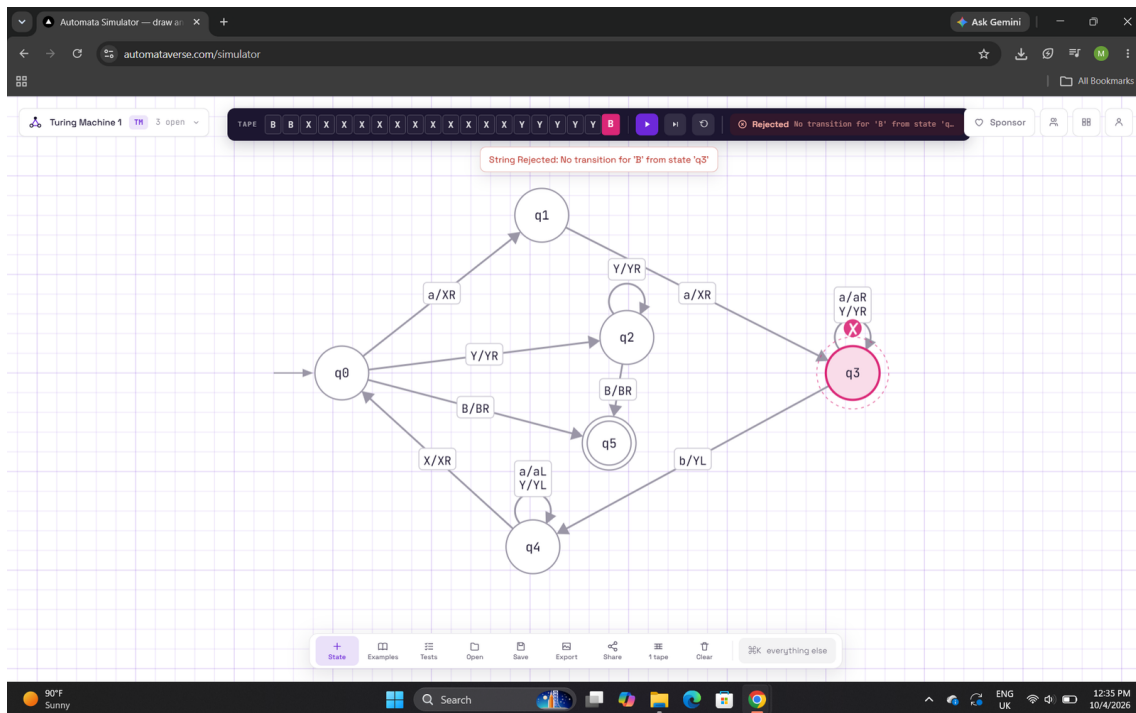
Repo files: q2_run1_n.json, q2_run1_n.png, README line "Q2 run 1"

JSON export (q2_run1_n.json):

```
{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 120,
      "y": 110,
      "label": "q0"
    },
    "q1": {
      "id": "q1",
      "x": 454.19319024664463,
      "y": -48.84383524496698,
      "label": "q1"
    },
    "q2": {
      "id": "q2",
      "x": 462.987844214765,
      "y": 90.7228355285234,
      "label": "q2"
    },
    "q3": {
      "id": "q3",
      "x": 612.1202555352347,
      "y": 140.8434631543625,
      "label": "q3"
    },
    "q4": {
      "id": "q4",
      "x": 382.94052339094037,
      "y": 364.04843709563784,
      "label": "q4"
    },
    "q5": {
      "id": "q5",
      "x": 338.5302034177854,
      "y": 221.39742000671123,
      "label": "q5"
    }
  },
  "transitionMap": {
    "q0": [
      {
        "label": "a/XR",
        "sourceid": "q0",
        "targetid": "q1"
      },
      {
        "label": "Y/YR",
        "sourceid": "q0",
        "targetid": "q2"
      },
      {
        "label": "B/BR",
        "sourceid": "q0",
        "targetid": "q5"
      },
      {
        "label": "X/XR",
        "sourceid": "q0",
        "targetid": "q4"
      }
    ],
    "q1": [
      {
        "label": "a/XR",
        "sourceid": "q1",
        "targetid": "q3"
      }
    ],
    "q2": [
      {
        "label": "Y/YR",
        "sourceid": "q2",
        "targetid": "q2"
      },
      {
        "label": "a/aR",
        "sourceid": "q2",
        "targetid": "q3"
      },
      {
        "label": "Y/YR",
        "sourceid": "q2",
        "targetid": "q3"
      }
    ],
    "q3": [
      {
        "label": "a/aR",
        "sourceid": "q3",
        "targetid": "q3"
      },
      {
        "label": "Y/YR",
        "sourceid": "q3",
        "targetid": "q3"
      }
    ],
    "q4": [
      {
        "label": "a/aL",
        "sourceid": "q4",
        "targetid": "q4"
      },
      {
        "label": "Y/YL",
        "sourceid": "q4",
        "targetid": "q4"
      }
    ],
    "q5": [
      {
        "label": "B/BR",
        "sourceid": "q5",
        "targetid": "q0"
      },
      {
        "label": "X/XR",
        "sourceid": "q5",
        "targetid": "q4"
      }
    ]
  },
  "finalNodes": [
    "q5"
  ],
  "nodeNum": 6,
  "autoMataType": "TM",
  "inputString": "aaaaaaaaaabbabbbb",
  "stageProps": {
    "x": 239.56762429952187,
    "y": 181.3041535406245,
    "scale": 1.2968712300500012,
    "draggable": true,
    "stateOutput": {},
    "tapeMode": "single",
    "inputString2": ""
  }
}
```

Run 2: 12 a's then 5 b's (one b removed)

Input: aaaaaaaaaabbbbb. Result: **Rejected** (stopped in q3, which found no b to match). Correct, 5 b's is not half of 12 a's.



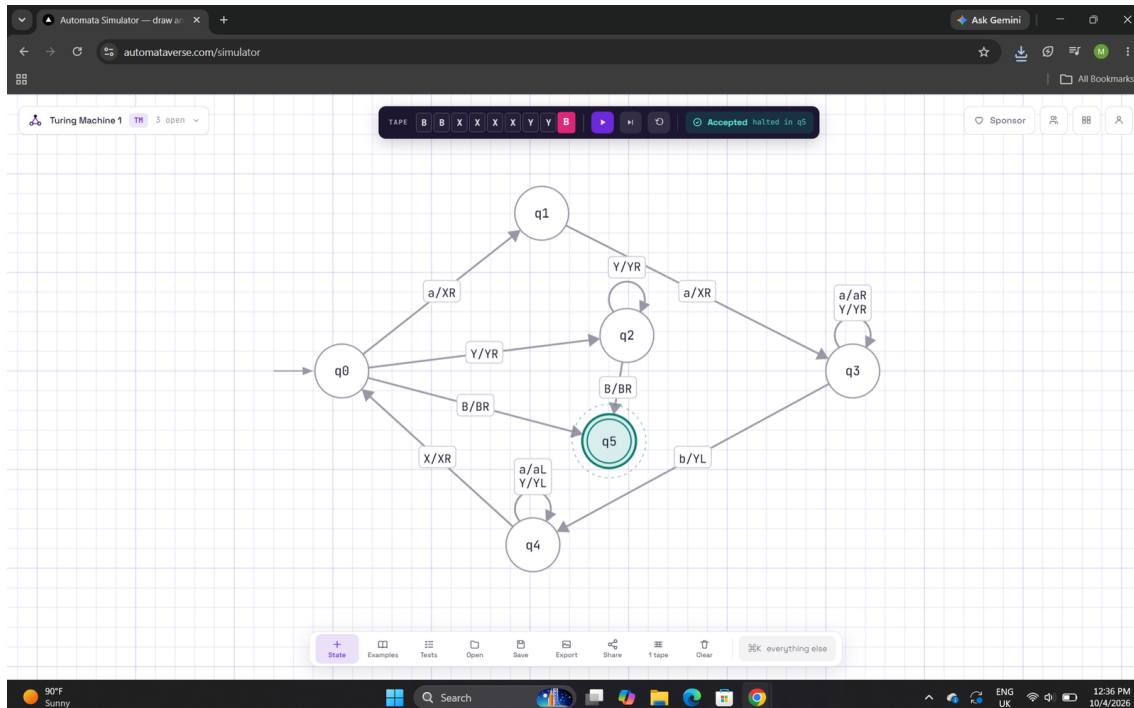
Repo files: q2_run2_off.json, q2_run2_off.png, README line "Q2 run 2"

JSON export (q2_run2_off.json):

```
{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 120,
      "y": 110,
      "label": "q0"
    },
    "q1": {
      "id": "q1",
      "x": 343,
      "y": 110,
      "label": "q1"
    },
    "q2": {
      "id": "q2",
      "x": 438,
      "y": 70,
      "label": "q2"
    },
    "q3": {
      "id": "q3",
      "x": 690,
      "y": 110,
      "label": "q3"
    },
    "q4": {
      "id": "q4",
      "x": 333,
      "y": 223,
      "label": "q4"
    },
    "q5": {
      "id": "q5",
      "x": 418,
      "y": 188,
      "label": "q5"
    }
  },
  "transitionMap": {
    "q0": [
      {
        "label": "a/XR",
        "sourceid": "q0",
        "targetid": "q1"
      },
      {
        "label": "Y/YR",
        "sourceid": "q0",
        "targetid": "q2"
      },
      {
        "label": "B/BR",
        "sourceid": "q0",
        "targetid": "q5"
      },
      {
        "label": "X/XR",
        "sourceid": "q0",
        "targetid": "q4"
      }
    ],
    "q1": [
      {
        "label": "a/XR",
        "sourceid": "q1",
        "targetid": "q3"
      },
      {
        "label": "Y/YR",
        "sourceid": "q1",
        "targetid": "q2"
      }
    ],
    "q2": [
      {
        "label": "a/aL",
        "sourceid": "q2",
        "targetid": "q4"
      },
      {
        "label": "Y/YL",
        "sourceid": "q2",
        "targetid": "q5"
      }
    ],
    "q3": [
      {
        "label": "a/aR",
        "sourceid": "q3",
        "targetid": "q3"
      },
      {
        "label": "Y/YR",
        "sourceid": "q3",
        "targetid": "q2"
      }
    ],
    "q4": [
      {
        "label": "a/aL",
        "sourceid": "q4",
        "targetid": "q0"
      },
      {
        "label": "Y/YL",
        "sourceid": "q4",
        "targetid": "q5"
      }
    ],
    "q5": [
      {
        "label": "B/BR",
        "sourceid": "q5",
        "targetid": "q0"
      },
      {
        "label": "Y/YR",
        "sourceid": "q5",
        "targetid": "q2"
      }
    ]
  },
  "finalNodes": [
    "q3"
  ],
  "nodeName": "q3",
  "nodeNum": 6,
  "automataType": "TM",
  "inputString": "aaaaaaaaabbbbb",
  "stageProps": {
    "x": 306,
    "y": 240,
    "scale": 1.21,
    "draggable": true
  },
  "stateOutput": {},
  "tapeMode": "single",
  "inputString2": ""
}
```

Run 3: aaaabb (my own string, k = 2)

Result: **Accepted** (halted in q5). Correct, 4 a's and 2 b's fits $a^{2k}b^k$ with $k = 2$.



Repo files: q2_run3_extra.json, q2_run3_extra.png, README line "Q2 run 3"

JSON export (q2_run3_extra.json):

```
{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 120,
      "y": 110,
      "label": "q0"
    },
    "q1": {
      "id": "q1",
      "x": 343.0578512396696,
      "y": -66.03305785123963,
      "label": "q1"
    },
    "q2": {
      "id": "q2",
      "x": 438.0165289256197,
      "y": 70.33057851239667,
      "label": "q2"
    },
    "q3": {
      "id": "q3",
      "x": 690,
      "y": 110,
      "label": "q3"
    },
    "q4": {
      "id": "q4",
      "x": 333.22314049586777,
      "y": 303.9669421487604,
      "label": "q4"
    },
    "q5": {
      "id": "q5",
      "x": 418.2644628099174,
      "y": 188.26446280991735,
      "label": "q5"
    }
  },
  "transitionMap": {
    "q0": [
      {
        "label": "a/XR",
        "sourceid": "q0",
        "targetid": "q1"
      },
      {
        "label": "Y/YR",
        "sourceid": "q0",
        "targetid": "q2"
      },
      {
        "label": "X/XR",
        "sourceid": "q0",
        "targetid": "q4"
      }
    ],
    "q1": [
      {
        "label": "Y/YR",
        "sourceid": "q1",
        "targetid": "q2"
      },
      {
        "label": "a/XR",
        "sourceid": "q1",
        "targetid": "q3"
      }
    ],
    "q2": [
      {
        "label": "Y/YR",
        "sourceid": "q2",
        "targetid": "q2"
      },
      {
        "label": "B/BR",
        "sourceid": "q2",
        "targetid": "q5"
      },
      {
        "label": "a/XR",
        "sourceid": "q2",
        "targetid": "q3"
      }
    ],
    "q3": [
      {
        "label": "a/aR, Y/YR",
        "sourceid": "q3",
        "targetid": "q3"
      },
      {
        "label": "b/YL",
        "sourceid": "q3",
        "targetid": "q4"
      }
    ],
    "q4": [
      {
        "label": "a/aL, Y/YL",
        "sourceid": "q4",
        "targetid": "q4"
      },
      {
        "label": "X/XR",
        "sourceid": "q4",
        "targetid": "q0"
      }
    ],
    "q5": [
      {
        "label": "Y/YR",
        "sourceid": "q5",
        "targetid": "q2"
      }
    ]
  },
  "finalNodes": [
    "q5"
  ],
  "nodeName": "q5",
  "automataType": "TM",
  "inputString": "aaaabb",
  "stageProps": {
    "x": 306.80999999999995,
    "y": 240.44999999999993,
    "scale": 1.2100000000000002,
    "draggable": true
  },
  "stateOutput": {},
  "tapeMode": "single",
  "inputString2": ""
}
```

Task D - Where an off-by-one error could happen

Off-by-one mistakes could happen in three places. In q1: without it, an odd number of a's like aaab would be accepted. In q4: the head has to stop on the last X and then step one cell right, otherwise q0 starts the next round on the wrong cell. In q2: without that check, extra b's at the end would be accepted. The run with 12 a's and 5 b's shows it. Five rounds use 10 a's and 5 b's, then the sixth round crosses off the last two a's and q3 finds no b, so it rejects. The final tape has 12 X and 5 Y, which is what I expected.

Q3 - Binary Transformer: x2 (V = 2, B = 10010)

Task A - Algorithm

Doubling a binary number just puts a 0 on the end (101 is 5 and 1010 is 10). So the machine goes to the end of the number and writes 0 on the first blank. q0 moves right over the 0s and 1s and leaves them alone. At the first blank it writes 0, moves left and goes to q1, which halts. Nothing else on the tape changes, and the answer is one cell longer, so no clean-up is needed.

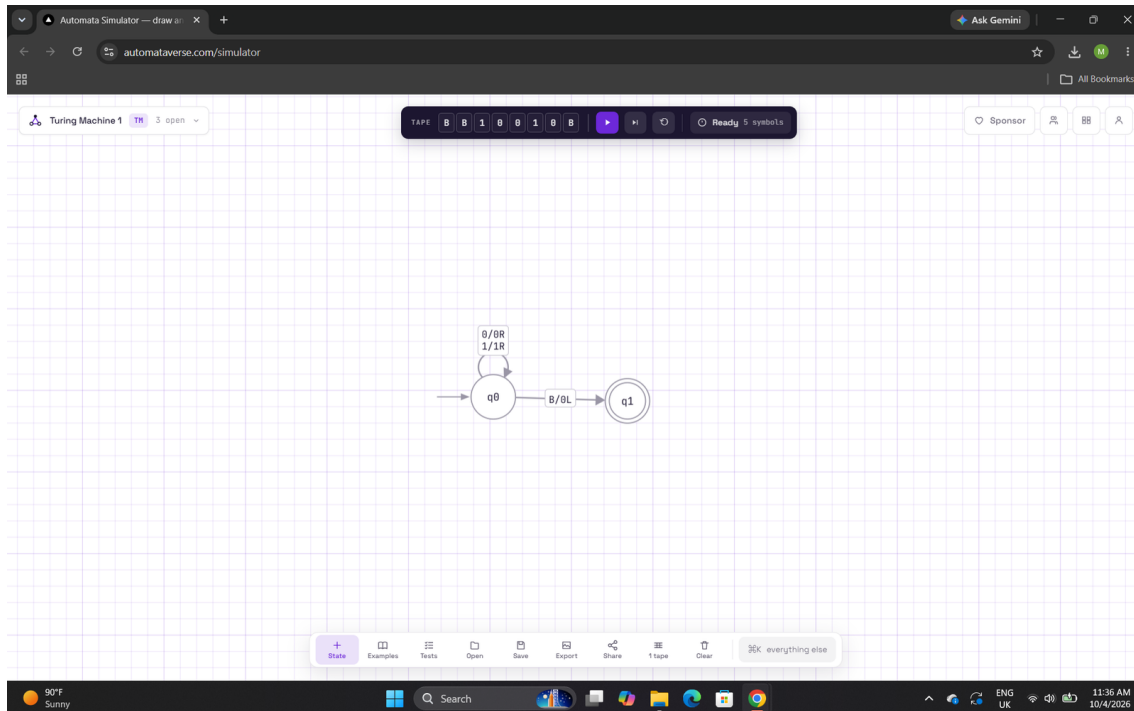
Task B - Transition table

Current state	Read	Write	Move	Next state
q0	0	0	R	q0
q0	1	1	R	q0
q0	B	0	L	q1
q1	-	-	-	accept state (halts, accepted)

Task C - Simulator run

Run 1 (before): start tape = my B = 10010

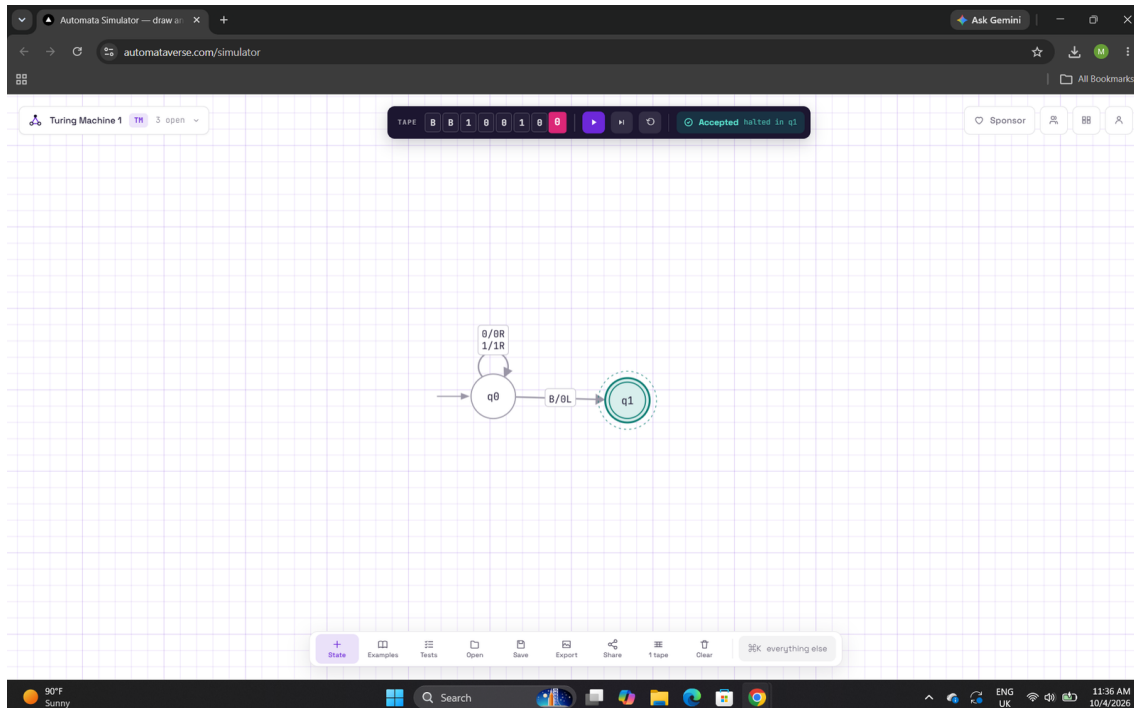
The tape holds 10010 and the machine is ready to run.



Repo files: q3_run1_before.png

Run 1 (after): halted

Result: halted in q1 (accepted) with the tape 100100.



Repo files: q3_run1_B.json, q3_run1_after.png, README line "Q3 run 1"

JSON export (q3_run1_B.json):

```
{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 334.4935017348487,
      "y": 118.7245197449817
    },
    "q1": {
      "id": "q1",
      "x": 515.5883487591425,
      "y": 124.14200694852093
    }
  },
  "transitionMap": {
    "q0": [
      {
        "label": "0/0R, 1/1R",
        "sourceid": "q0",
        "targetid": "q0"
      },
      {
        "label": "B/0L",
        "sourceid": "q0",
        "targetid": "q1"
      }
    ],
    "finalNodes": [
      "q1"
    ],
    "nodeName": 2,
    "automataType": "TM",
    "inputString": "10010",
    "stageProps": {
      "x": 322,
      "y": 289,
      "scale": 1,
      "draggable": true,
      "stateOutput": {},
      "tapeMode": "single",
      "inputString2": ""
    }
  }
}
```

By hand: $B = 10010 = 1 \times 16 + 0 \times 8 + 0 \times 4 + 1 \times 2 + 0 \times 1 = 18$. Doubling gives $18 \times 2 = 36$. The final tape $100100 = 1 \times 32 + 0 \times 16 + 0 \times 8 + 1 \times 4 + 0 \times 2 + 0 \times 1 = 36$. So the final tape matches the value it should give. (My operation is $\times 2$, not decrement, so no change to B was needed.)

Task D - Edge case

The head has to go all the way to the right and write on the first blank. If it only changed the leftmost bit, or put the 0 on top of the last bit, the answer would be wrong. For example 10010 would turn into 10000, which is 16, not 36. Short inputs still work: 0 becomes 00 (still 0) and an empty tape becomes 0.

Q4 - Even-length palindromes over {a,b} (V = 2, n = 6)

Task A - Strategy

The language is a string followed by its reverse, so it has to be a palindrome of even length. The machine erases the first symbol and remembers it: q0 goes to q1 if it was a and to q2 if it was b. That state walks right to the end and steps back one, and then q3 (for a) or q4 (for b) checks that the last symbol is the same letter and erases it. q5 goes back left to the blank and steps into q0 for the next pair. If q0 sees a blank, every pair matched, so it goes to q6 and accepts. The even length comes from q3 and q4: if they find a blank where the last symbol should be, the first symbol had no partner, the length was odd, and it rejects. The empty string is accepted right away in q0.

Task B - Transition table

Current state	Read	Write	Move	Next state
q0	a	B	R	q1
q0	b	B	R	q2
q0	B	B	R	q6
q1	a	a	R	q1
q1	b	b	R	q1
q1	B	B	L	q3
q2	a	a	R	q2
q2	b	b	R	q2
q2	B	B	L	q4
q3	a	B	L	q5
q3	any other: B b	same	R	no transition: halts and rejects
q4	b	B	L	q5
q4	any other: B a	same	R	no transition: halts and rejects
q5	a	a	L	q5
q5	b	b	L	q5
q5	B	B	R	q0
q6	-	-	-	accept state (halts, accepted)

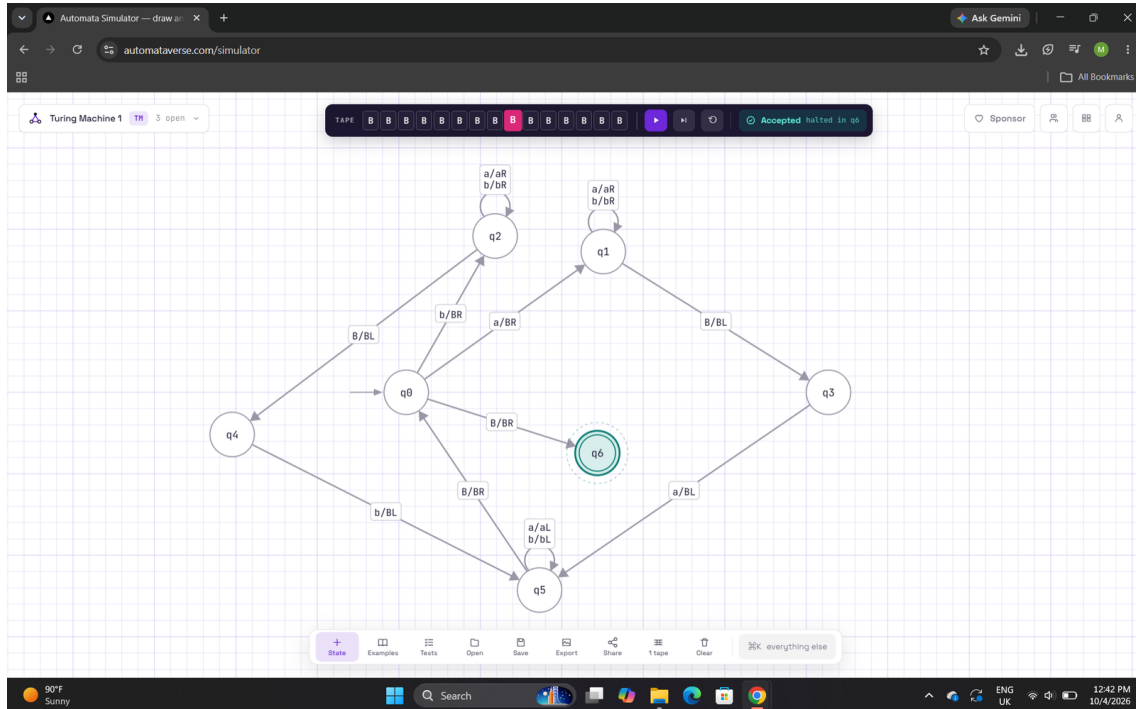
What each state does: q0 = take the leftmost symbol. q1 / q2 = carry an a / a b to the right end. q3 / q4 = compare with the last symbol (a / b). q5 = go back to the left boundary. q6 = accept.

Task C - Simulator runs

The machine erases symbols as it matches them, so an accepted string ends with an all-blank tape.

Run 1: aabbabbabbaa (length 12 = 2n, it is aabbab followed by its reverse babbaa)

Result: Accepted (halted in q6, tape fully erased). Correct, the string is an even palindrome.



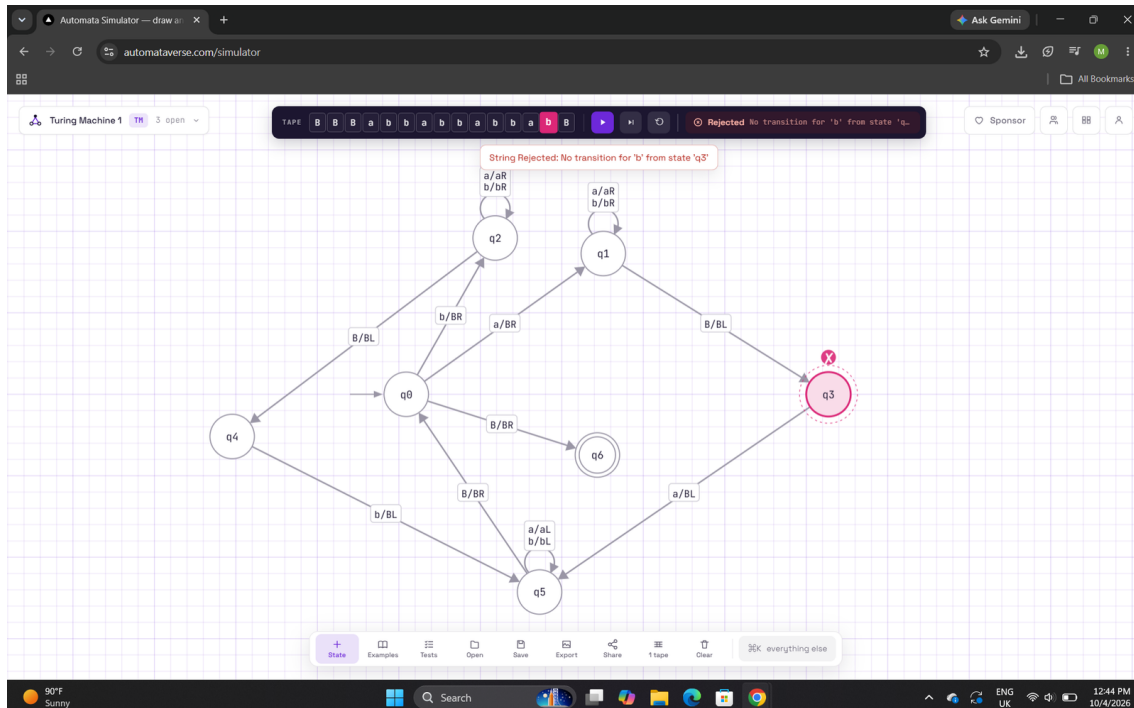
Repo files: q4_run1_in.json, q4_run1_in.png, README line "Q4 run 1"

JSON export (q4_run1_in.json):

```
{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 120,
      "y": 110
    },
    "q1": {
      "id": "q1",
      "x": 386,
      "y": -80
    },
    "q2": {
      "id": "q2",
      "x": 240,
      "y": -101
    },
    "q3": {
      "id": "q3",
      "x": 690,
      "y": 110
    },
    "q4": {
      "id": "q4",
      "x": -115,
      "y": 167
    },
    "q5": {
      "id": "q5",
      "x": 300,
      "y": 377
    },
    "q6": {
      "id": "q6",
      "x": 378,
      "y": 192
    }
  },
  "transitionMap": {
    "q0": [
      {
        "label": "a/BR",
        "sourceid": "q0",
        "targetid": "q1"
      },
      {
        "label": "b/BR",
        "sourceid": "q0",
        "targetid": "q6"
      },
      {
        "label": "B/BL",
        "sourceid": "q0",
        "targetid": "q4"
      },
      {
        "label": "B/BR",
        "sourceid": "q0",
        "targetid": "q5"
      }
    ],
    "q1": [
      {
        "label": "a/aR, b/bR",
        "sourceid": "q1",
        "targetid": "q1"
      },
      {
        "label": "B/BL",
        "sourceid": "q1",
        "targetid": "q3"
      }
    ],
    "q2": [
      {
        "label": "a/aR, b/bR",
        "sourceid": "q2",
        "targetid": "q2"
      },
      {
        "label": "B/BL",
        "sourceid": "q2",
        "targetid": "q4"
      }
    ],
    "q3": [
      {
        "label": "a/BL",
        "sourceid": "q3",
        "targetid": "q5"
      }
    ],
    "q4": [
      {
        "label": "b/BL",
        "sourceid": "q4",
        "targetid": "q5"
      }
    ],
    "q5": [
      {
        "label": "a/aL, b/bL",
        "sourceid": "q5",
        "targetid": "q5"
      },
      {
        "label": "B/BR",
        "sourceid": "q5",
        "targetid": "q0"
      }
    ],
    "q6": [
      {
        "label": "a/aL, b/bL",
        "sourceid": "q6",
        "targetid": "q6"
      }
    ]
  },
  "finalNodes": [
    "q6"
  ],
  "nodeNum": 7,
  "automataType": "TM",
  "inputString": "aabbabbabbaa",
  "stageProps": {
    "x": 419,
    "y": 295,
    "scale": 1,
    "draggable": true,
    "stateOutput": {},
    "tapeMode": "single",
    "inputString2": ""
  }
}
```

Run 2: aabbabbabbab (length 12, last symbol differs from the first)

Result: **Rejected** (stopped in q3, which was waiting for an a but found a b). Correct, it is not a palindrome.



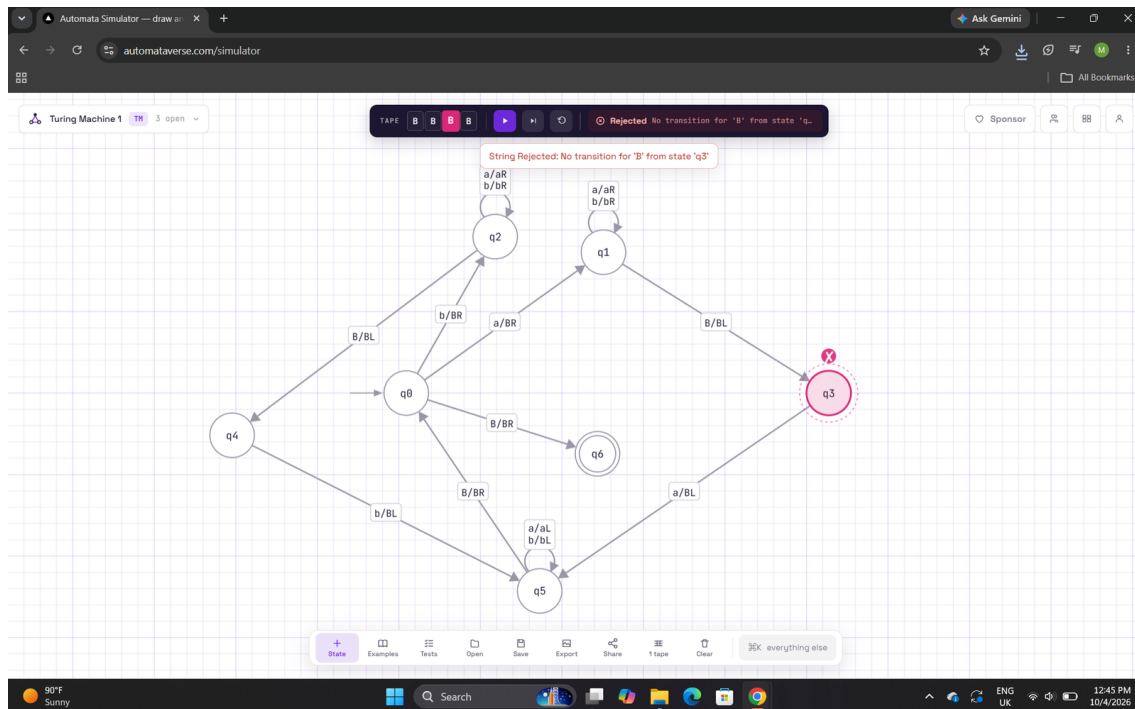
Repo files: q4_run2_notin.json, q4_run2_notin.png, README line "Q4 run 2"

JSON export (q4_run2_notin.json):

```
{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 120,
      "y": 110
    },
    "q1": {
      "id": "q1",
      "x": 386,
      "y": -80
    },
    "q2": {
      "id": "q2",
      "x": 240,
      "y": -101
    },
    "q3": {
      "id": "q3",
      "x": 690,
      "y": 110
    },
    "q4": {
      "id": "q4",
      "x": -115,
      "y": 167
    },
    "q5": {
      "id": "q5",
      "x": 300,
      "y": 377
    },
    "q6": {
      "id": "q6",
      "x": 378,
      "y": 192
    }
  },
  "transitionMap": {
    "q0": [
      {
        "label": "a/BR",
        "sourceid": "q0",
        "targetid": "q1"
      },
      {
        "label": "b/BR",
        "sourceid": "q0",
        "targetid": "q6"
      },
      {
        "label": "B/BL",
        "sourceid": "q0",
        "targetid": "q4"
      },
      {
        "label": "B/BR",
        "sourceid": "q0",
        "targetid": "q5"
      }
    ],
    "q1": [
      {
        "label": "a/aR,b/bR",
        "sourceid": "q1",
        "targetid": "q1"
      },
      {
        "label": "B/BL",
        "sourceid": "q1",
        "targetid": "q3"
      }
    ],
    "q2": [
      {
        "label": "a/aR,b/bR",
        "sourceid": "q2",
        "targetid": "q2"
      },
      {
        "label": "B/BL",
        "sourceid": "q2",
        "targetid": "q4"
      }
    ],
    "q3": [
      {
        "label": "a/BL",
        "sourceid": "q3",
        "targetid": "q5"
      }
    ],
    "q4": [
      {
        "label": "b/BL",
        "sourceid": "q4",
        "targetid": "q5"
      }
    ],
    "q5": [
      {
        "label": "a/aL,b/bL",
        "sourceid": "q5",
        "targetid": "q5"
      },
      {
        "label": "B/BR",
        "sourceid": "q5",
        "targetid": "q0"
      }
    ],
    "q6": [
      {
        "label": "B/BR",
        "sourceid": "q6",
        "targetid": "q0"
      }
    ]
  },
  "finalNodes": [
    "q6"
  ],
  "nodeNum": 7,
  "automataType": "TM",
  "inputString": "aabbabbabbab",
  "stageProps": {
    "x": 419,
    "y": 295,
    "scale": 1,
    "draggable": true,
    "stateOutput": {},
    "tapeMode": "single",
    "inputString2": ""
  }
}
```

Run 3: a (a single symbol, the boundary case for an even-only language)

Result: **Rejected** (stopped in q3 on a blank). Correct, a single symbol is a palindrome but has odd length, so it is not in my language.



Repo files: q4_run3_boundary.json, q4_run3_boundary.png, README line "Q4 run 3"

JSON export (q4_run3_boundary.json):

```
{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 120,
      "y": 110
    },
    "q1": {
      "id": "q1",
      "x": 386,
      "y": -80
    },
    "q2": {
      "id": "q2",
      "x": 240,
      "y": -101
    },
    "q3": {
      "id": "q3",
      "x": 690,
      "y": 110
    },
    "q4": {
      "id": "q4",
      "x": -115,
      "y": 167
    },
    "q5": {
      "id": "q5",
      "x": 300,
      "y": 377
    },
    "q6": {
      "id": "q6",
      "x": 378,
      "y": 192
    }
  },
  "transitionMap": {
    "q0": [
      {
        "label": "a/BR",
        "sourceid": "q0",
        "targetid": "q1"
      },
      {
        "label": "b/BR",
        "sourceid": "q0",
        "targetid": "q2"
      },
      {
        "label": "B/BL",
        "sourceid": "q0",
        "targetid": "q4"
      },
      {
        "label": "B/BR",
        "sourceid": "q0",
        "targetid": "q6"
      }
    ],
    "q1": [
      {
        "label": "a/aR, b/bR",
        "sourceid": "q1",
        "targetid": "q2"
      },
      {
        "label": "B/BL",
        "sourceid": "q1",
        "targetid": "q3"
      }
    ],
    "q2": [
      {
        "label": "a/aR, b/bR",
        "sourceid": "q2",
        "targetid": "q1"
      },
      {
        "label": "B/BL",
        "sourceid": "q2",
        "targetid": "q4"
      }
    ],
    "q3": [
      {
        "label": "a/aL, b/bL",
        "sourceid": "q3",
        "targetid": "q5"
      }
    ],
    "q4": [
      {
        "label": "B/BL",
        "sourceid": "q4",
        "targetid": "q5"
      }
    ],
    "q5": [
      {
        "label": "a/aL, b/bL",
        "sourceid": "q5",
        "targetid": "q5"
      },
      {
        "label": "B/BR",
        "sourceid": "q5",
        "targetid": "q6"
      }
    ]
  },
  "finalNodes": [
    "q6"
  ],
  "nodeName": "q6",
  "nodeNum": 7,
  "automataType": "TM",
  "inputString": "a",
  "stageProps": {
    "x": 419,
    "y": 295,
    "scale": 1,
    "draggable": true,
    "stateOutput": {},
    "tapeMode": "single",
    "inputString2": ""
  }
}
```

Task D - Why one pass is not enough

One symbol is all the machine can remember in its state (q1 or q2), so one trip checks one pair. It carries the first symbol to the far end, compares, and comes back using q5 to get the next pair. That's why every pair needs a full sweep. With one pass it would have to remember the whole first half, which could be any length, and it only has a few states. The blanks left behind by the erasing are how it keeps track of where it is.

Q5 - $a^i \# b^j$ with $i > j$ ($V = 2$, $B = 10010$)

Task A - Strategy

Each round pairs one a on the left of the # with one b on the right. q0 skips X's and turns the first a into X, q1 goes right to the #, q2 skips Y's and turns the first b into Y, and q3 goes back left to the last X and steps right into q0. If q2 reaches a blank without finding a b, the a it just marked has no b left, so there are more a's than b's and the machine accepts. If q0 reaches the # before it finds an a, the a's ran out first or tied, so it rejects. Anything in the wrong place (a b before the # or an a after it) leaves no transition, so those get rejected too.

Task B - Transition table

Current state	Read	Write	Move	Next state
q0	X	X	R	q0
q0	a	X	R	q1
q0	any other: # B Y b	same	R	no transition: halts and rejects
q1	a	a	R	q1
q1	#	#	R	q2
q1	any other: B X Y b	same	R	no transition: halts and rejects
q2	Y	Y	R	q2
q2	b	Y	L	q3
q2	B	B	R	q4
q2	any other: # X a	same	R	no transition: halts and rejects
q3	Y	Y	L	q3
q3	#	#	L	q3
q3	a	a	L	q3
q3	X	X	R	q0
q3	any other: B b	same	R	no transition: halts and rejects
q4	-	-	-	accept state (halts, accepted)

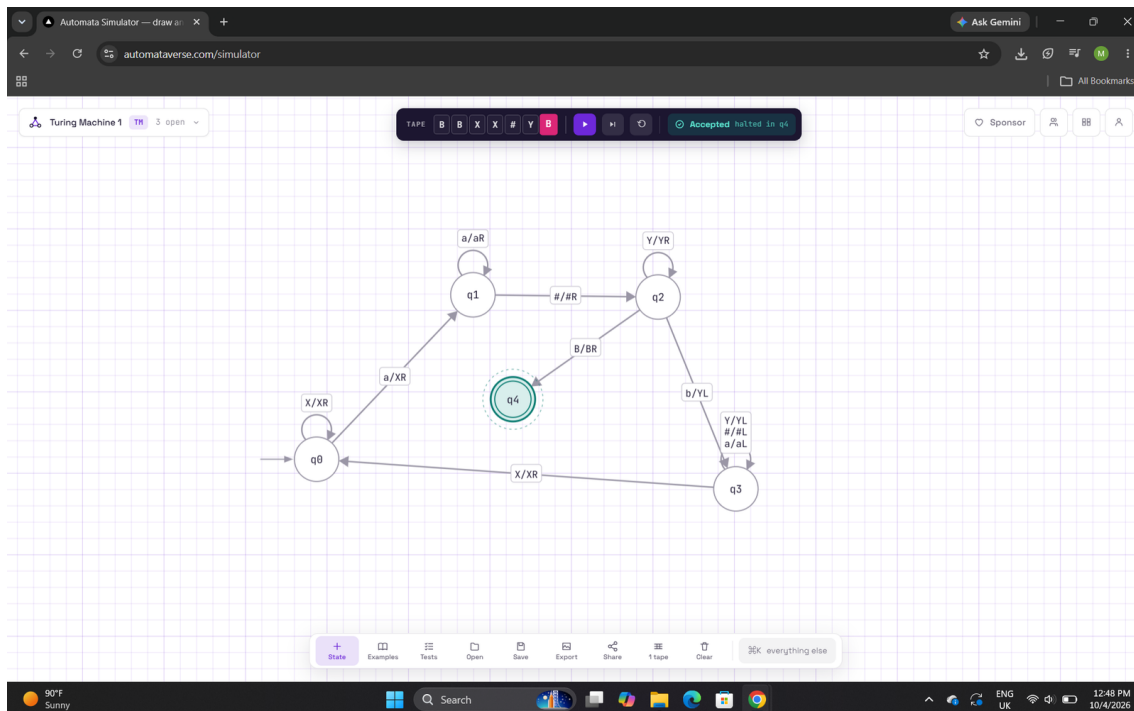
What each state does: q0 = find the next unmarked a. q1 = go right to the #. q2 = find an unmarked b (a blank here means accept). q3 = go back left to the last X. q4 = accept.

Task C - Test strings and simulator runs

How I converted B. B = 10010 has two 1-bits and three 0-bits. I wrote one a for each 1-bit, so the left side is aa ($i = 2$). The language only allows a's before the #, so I did not put the b's from the 0-bits on the left side. The right side is chosen for each run below.

Run 1 (member): aa#b

$i = 2$ and $j = 1$, so $i > j$. Result: **Accepted** (halted in q_4). Correct, the string is in the language. The tape ends as $XX\#Y$.



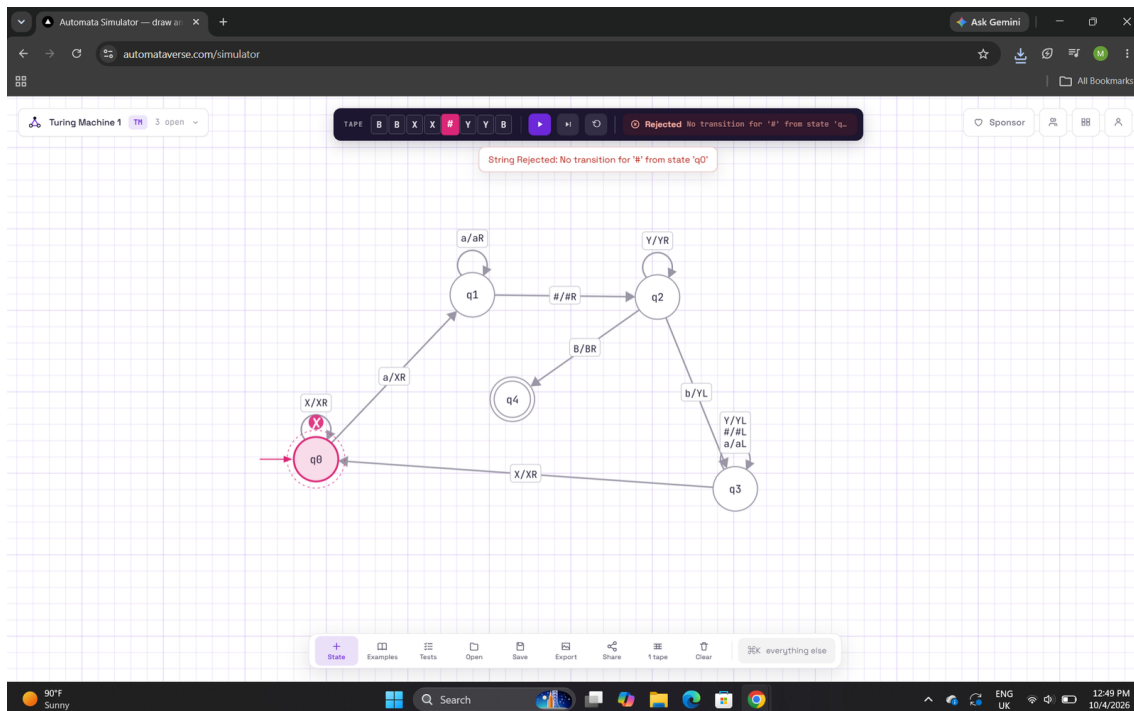
Repo files: q5_run1_member.json, q5_run1_member.png, README line "Q5 run 1"

JSON export (q5_run1_member.json):

```
{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 96,
      "y": 201
    },
    "q1": {
      "id": "q1",
      "x": 307,
      "y": -21
    },
    "q2": {
      "id": "q2",
      "x": 557,
      "y": -18
    },
    "q3": {
      "id": "q3",
      "x": 662,
      "y": 241
    },
    "q4": {
      "id": "q4",
      "x": 361,
      "y": 120
    }
  },
  "transitionMap": {
    "q0": [
      {
        "label": "X/XR",
        "sourceid": "q0",
        "targetid": "q0"
      },
      {
        "label": "a/aR",
        "sourceid": "q0",
        "targetid": "q1"
      }
    ],
    "q1": [
      {
        "label": "a/aR",
        "sourceid": "q1",
        "targetid": "q1"
      },
      {
        "label": "#/#R",
        "sourceid": "q1",
        "targetid": "q2"
      }
    ],
    "q2": [
      {
        "label": "Y/YR",
        "sourceid": "q2",
        "targetid": "q2"
      },
      {
        "label": "B/BR",
        "sourceid": "q2",
        "targetid": "q4"
      },
      {
        "label": "b/YL",
        "sourceid": "q2",
        "targetid": "q3"
      }
    ],
    "q3": [
      {
        "label": "Y/YL, #/#L, a/aL",
        "sourceid": "q3",
        "targetid": "q3"
      },
      {
        "label": "X/XR",
        "sourceid": "q3",
        "targetid": "q0"
      }
    ]
  },
  "finalNodes": [
    "q4"
  ],
  "nodeNum": 5,
  "automataType": "TM",
  "inputString": "aa#b",
  "stageProps": {
    "x": 322,
    "y": 289,
    "scale": 1,
    "draggable": true
  },
  "stateOutput": {},
  "tapeMode": "single",
  "inputString2": ""
}
```

Run 2 (not a member): aa#bb

$i = 2$ and $j = 2$, so i is not bigger than j . Result: **Rejected** (q_0 reached the $\#$ with no a left, so it has no transition). Correct, the string is not in the language.



Repo files: q5_run2_nonmember.json, q5_run2_nonmember.png, README line "Q5 run 2"

JSON export (q5_run2_nonmember.json):

```
{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 96,
      "y": 201
    },
    "q1": {
      "id": "q1",
      "x": 307,
      "y": -21
    },
    "q2": {
      "id": "q2",
      "x": 557,
      "y": -18
    },
    "q3": {
      "id": "q3",
      "x": 662,
      "y": 241
    },
    "q4": {
      "id": "q4",
      "x": 361,
      "y": 120
    }
  },
  "transitionMap": {
    "q0": [
      {
        "label": "X/XR",
        "sourceid": "q0",
        "targetid": "q1"
      }
    ],
    "q1": [
      {
        "label": "a/aR",
        "sourceid": "q1",
        "targetid": "q1"
      },
      {
        "label": "#/BR",
        "sourceid": "q1",
        "targetid": "q2"
      }
    ],
    "q2": [
      {
        "label": "Y/YR",
        "sourceid": "q2",
        "targetid": "q2"
      },
      {
        "label": "B/BR",
        "sourceid": "q2",
        "targetid": "q4"
      },
      {
        "label": "b/YL",
        "sourceid": "q2",
        "targetid": "q3"
      }
    ],
    "q3": [
      {
        "label": "Y/YL, #/BL, a/aL",
        "sourceid": "q3",
        "targetid": "q3"
      },
      {
        "label": "X/XR",
        "sourceid": "q3",
        "targetid": "q0"
      }
    ]
  },
  "finalNodes": ["q4"],
  "nodeNum": 5,
  "automataType": "TM",
  "inputString": "aa#bb",
  "stageProps": {
    "x": 322,
    "y": 289,
    "scale": 1,
    "draggable": true
  },
  "stateOutput": {},
  "tapeMode": "single",
  "inputString2": ""
}
```

Task D - Why one pass is not enough

To check $i > j$ it has to compare two counts and i can be any size. A head that only goes right with a few states can't remember that. So the count is kept on the tape: each X and its Y make one matched pair. The head can only be in one place, so for each pair it has to cross the $\#$ to the b side and come back (q_1 and q_2 going right, q_3 coming back). That's why it moves back and forth.

Q6 - Debug the Broken Machine

In my simulator the states are named $q_0 = q_{\text{even}}$, $q_1 = q_{\text{odd}}$, $q_2 = q_{\text{accept}}$ and $q_3 = q_{\text{reject}}$. In the hand traces I use the names from the assignment.

Task A - Hand traces of the machine as given

Input "" (empty string)

Step	State	Tape (head in brackets)	Head position
0	q_{even}	[B]	0
1	q_{odd}	B [B]	1
2	q_{reject}	B B [B]	2

The machine halts in q_{reject} . The empty string has zero 1s, which is even, so it should have been accepted.

Input "11"

Step	State	Tape (head in brackets)	Head position
0	q_{even}	[1] 1	0
1	q_{odd}	1 [1]	1
2	q_{even}	1 1 [B]	2
3	q_{odd}	1 1 B [B]	3
4	q_{reject}	1 1 B B [B]	4

The machine halts in q_{reject} . The string has two 1s, which is even, so it should have been accepted.

Task B - Which row is wrong

The wrong row is (q_{even} , B) -> (B, R, q_{odd}). With the empty string the machine starts in q_{even} , sees a blank straight away, goes to q_{odd} and rejects. With 11 it's back in q_{even} after the two 1s, which is right, but the same blank row sends it to q_{odd} and it rejects as well. Both strings have an even number of 1s, so both should be accepted. Also nothing in the table goes to q_{accept} , so the machine could never accept anything. (q_{odd} , B) is fine, since ending in q_{odd} means an odd number of 1s. **The fix is (q_{even} , B) -> (B, R, q_{accept}).**

Broken machine as I entered it (wrong row highlighted):

Current state	Read	Write	Move	Next state
q_0	0	0	R	q_0
q_0	1	1	R	q_1
q_0	B	B	R	q_1
q_1	0	0	R	q_1
q_1	1	1	R	q_0
q_1	B	B	R	q_3
q_2	-	-	-	accept state (halts, accepted)
q_3	-	-	-	reject state (halts, rejected)

Corrected machine (only the highlighted row changed):

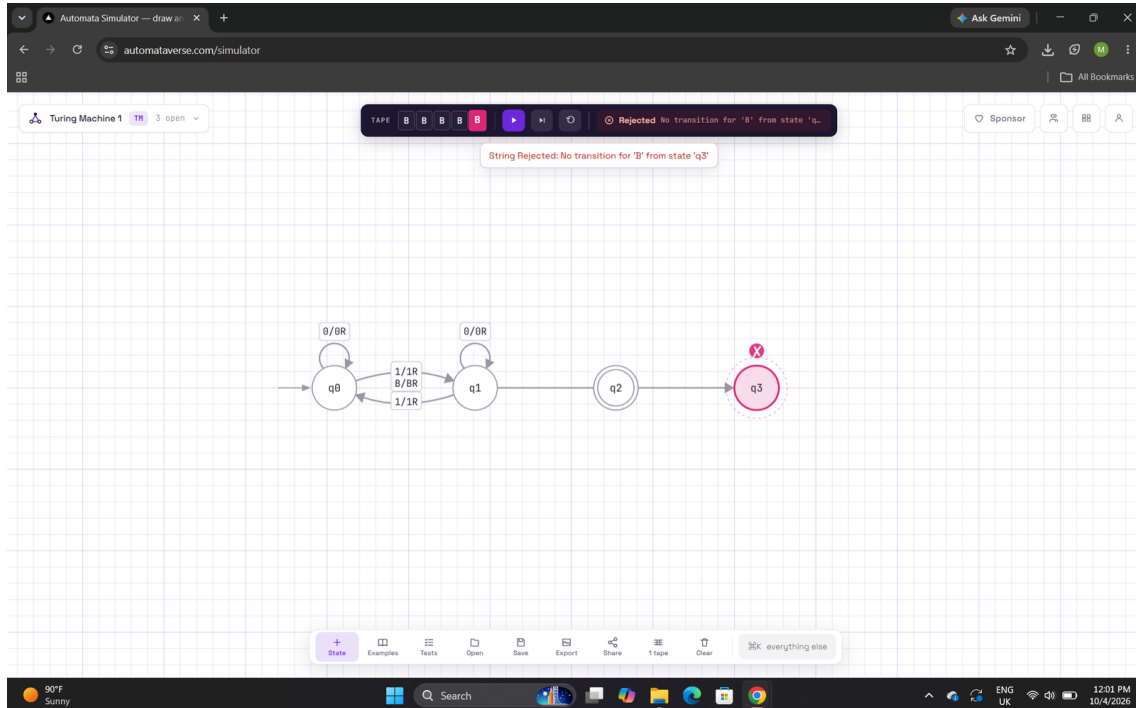
Current state	Read	Write	Move	Next state
q0	0	0	R	q0
q0	1	1	R	q1
q0	B	B	R	q2
q1	0	0	R	q1
q1	1	1	R	q0
q1	B	B	R	q3
q2	-	-	-	accept state (halts, accepted)
q3	-	-	-	reject state (halts, rejected)

Task C - Simulator runs (8 runs)

In the screenshots the arrow with the B/BR label goes behind the middle states because of how the states are laid out in the simulator, so the labels of those long arrows are hidden. They are the rows (q0, B) and (q1, B) in the tables above.

Run 1: broken machine on "" (empty string)

Result: **Rejected**. Correct answer: **Accepted**. This is WRONG, it shows the bug.



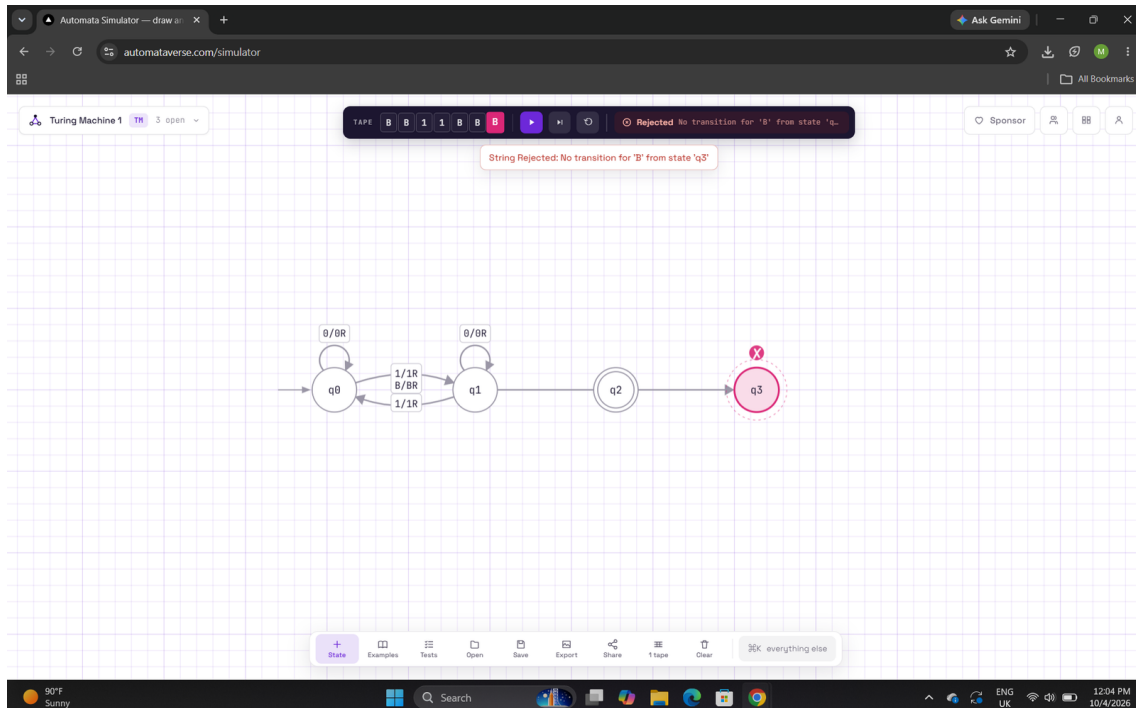
Repo files: q6_run1_broken_empty.json, q6_run1_broken_empty.png, README line "Q6 run 1"

JSON export (q6_run1_broken_empty.json):

```
{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 120,
      "y": 110
    },
    "q1": {
      "id": "q1",
      "x": 310,
      "y": 110
    },
    "q2": {
      "id": "q2",
      "x": 500,
      "y": 110
    },
    "q3": {
      "id": "q3",
      "x": 690,
      "y": 110
    }
  },
  "transitionMap": {
    "q0": [
      {
        "label": "0/0R",
        "sourceid": "q0",
        "targetid": "q0"
      },
      {
        "label": "1/1R, B/BR",
        "sourceid": "q0",
        "targetid": "q1"
      }
    ],
    "q1": [
      {
        "label": "0/0R",
        "sourceid": "q1",
        "targetid": "q1"
      },
      {
        "label": "1/1R",
        "sourceid": "q1",
        "targetid": "q0"
      },
      {
        "label": "B/BR",
        "sourceid": "q1",
        "targetid": "q3"
      }
    ],
    "q2": [
      {
        "label": "B/BR",
        "sourceid": "q2",
        "targetid": "q3"
      }
    ]
  },
  "finalNodes": [
    "q3"
  ],
  "nodeName": "q3",
  "nodeNum": 4,
  "automataType": "TM",
  "inputString": "",
  "stageProps": {
    "x": 322,
    "y": 289,
    "scale": 1,
    "draggable": true
  },
  "stateOutput": {},
  "tapeMode": "single",
  "inputString2": ""
}
```

Run 2: broken machine on "11"

Result: **Rejected**. Correct answer: **Accepted**. This is WRONG, it shows the bug.



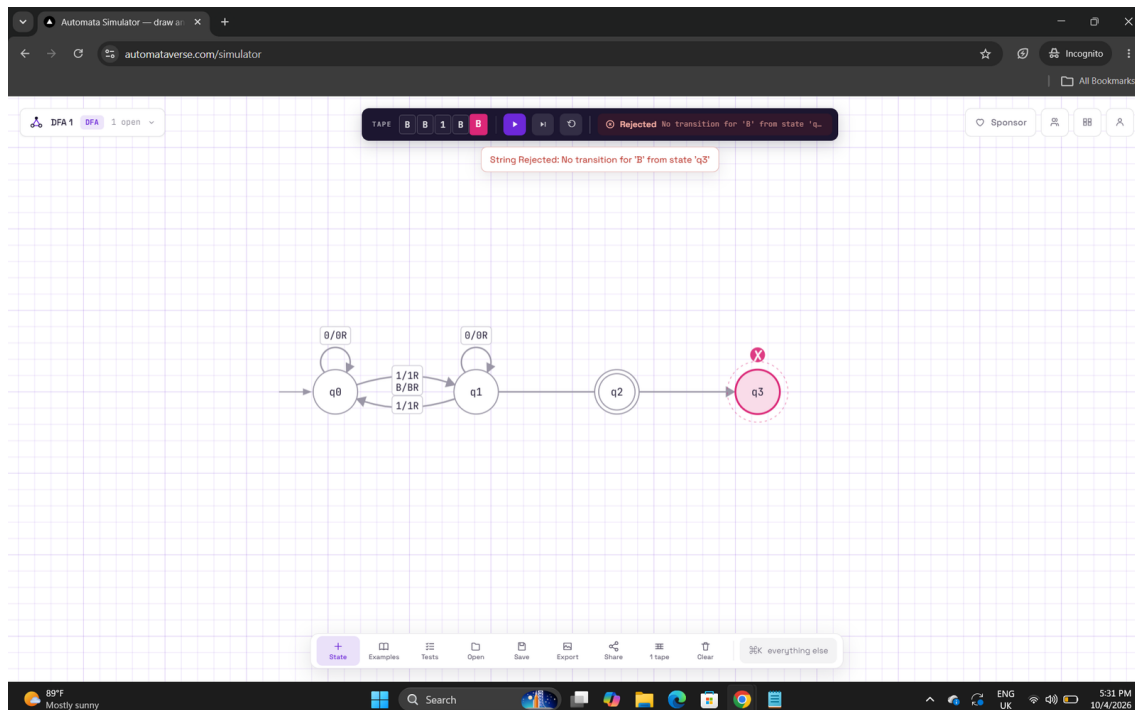
Repo files: q6_run2_broken_11.json, q6_run2_broken_11.png, README line "Q6 run 2"

JSON export (q6_run2_broken_11.json):

```
{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 120,
      "y": 110
    },
    "q1": {
      "id": "q1",
      "x": 310,
      "y": 110
    },
    "q2": {
      "id": "q2",
      "x": 500,
      "y": 110
    },
    "q3": {
      "id": "q3",
      "x": 690,
      "y": 110
    }
  },
  "transitionMap": {
    "q0": [
      {
        "label": "0/0R",
        "sourceid": "q0",
        "targetid": "q0",
        "label": "1/1R,B/BR",
        "sourceid": "q0",
        "targetid": "q1"
      }
    ],
    "q1": [
      {
        "label": "0/0R",
        "sourceid": "q1",
        "targetid": "q1",
        "label": "1/1R",
        "sourceid": "q1",
        "targetid": "q0",
        "label": "B/BR",
        "sourceid": "q1",
        "targetid": "q3"
      }
    ],
    "finalNodes": [
      "q2"
    ],
    "nodeName": 4,
    "automataType": "TM",
    "inputString": "11",
    "stageProps": {
      "x": 322,
      "y": 289,
      "scale": 1,
      "draggable": true
    },
    "stateOutput": {},
    "tapeMode": "single",
    "inputString2": ""
  }
}
```

Run 3: broken machine on "1"

Result: **Rejected**. Correct answer: **Rejected**. This matches.



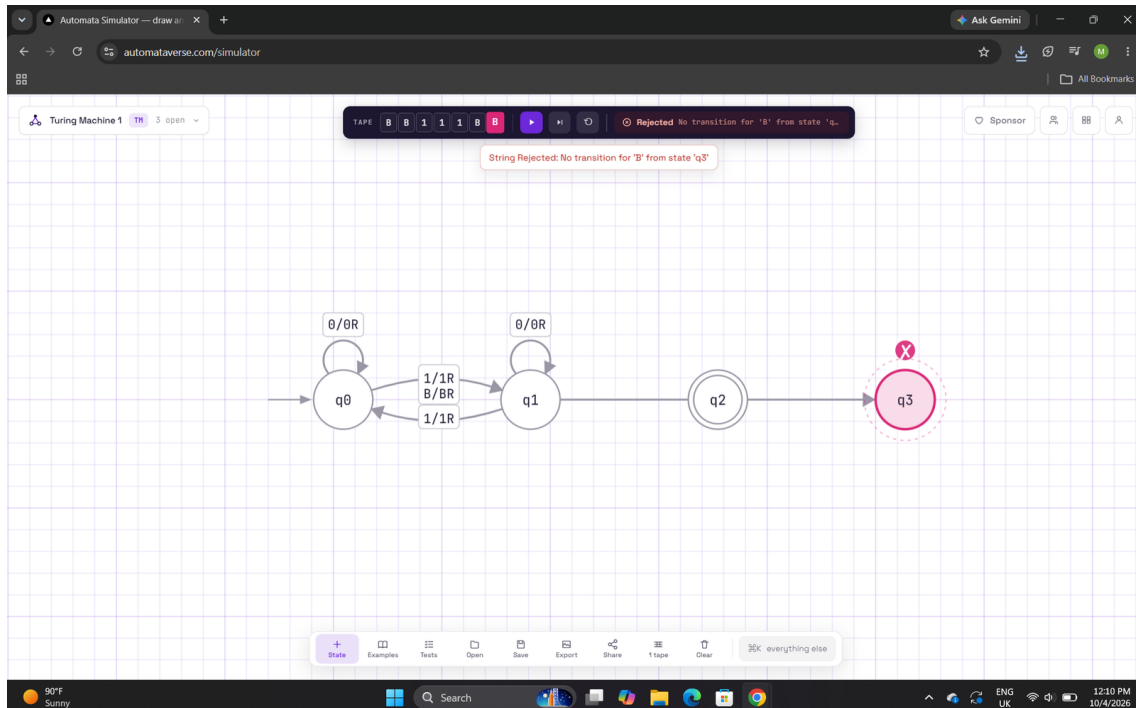
Repo files: q6_run3_broken_1.json, q6_run3_broken_1.png, README line "Q6 run 3"

JSON export (q6_run3_broken_1.json):

```
{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 120,
      "y": 110
    },
    "q1": {
      "id": "q1",
      "x": 310,
      "y": 110
    },
    "q2": {
      "id": "q2",
      "x": 500,
      "y": 110
    },
    "q3": {
      "id": "q3",
      "x": 690,
      "y": 110
    }
  },
  "transitionMap": {
    "q0": [
      {
        "label": "0/0R",
        "sourceid": "q0",
        "targetid": "q0",
        "label": "1/1R,B/BR",
        "sourceid": "q0",
        "targetid": "q1"
      },
      {
        "label": "0/0R",
        "sourceid": "q1",
        "targetid": "q1",
        "label": "1/1R",
        "sourceid": "q1",
        "targetid": "q0"
      },
      {
        "label": "B/BR",
        "sourceid": "q1",
        "targetid": "q3"
      }
    ],
    "finalNodes": [
      "q2"
    ],
    "nodeName": 4,
    "automataType": "TM",
    "inputString": "1",
    "stageProps": {
      "x": 322,
      "y": 289,
      "scale": 1,
      "draggable": true,
      "stateOutput": {},
      "tapeMode": "single",
      "inputString2": ""
    }
  }
}
```

Run 4: broken machine on "111"

Result: **Rejected**. Correct answer: **Rejected**. This matches.

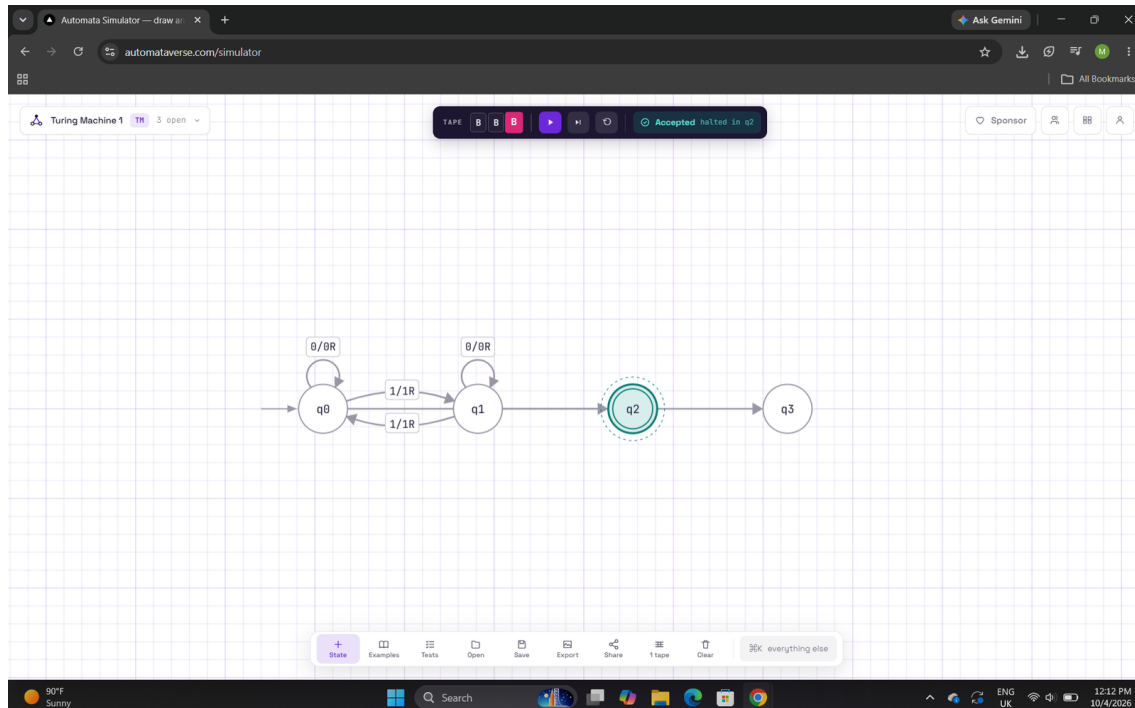


Repo files: q6_run4_broken_111.json, q6_run4_broken_111.png, README line "Q6 run 4"

JSON export (q6_run4_broken_111.json):

```
{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 120,
      "y": 110
    },
    "q1": {
      "id": "q1",
      "x": 310,
      "y": 110
    },
    "q2": {
      "id": "q2",
      "x": 500,
      "y": 110
    },
    "q3": {
      "id": "q3",
      "x": 690,
      "y": 110
    }
  },
  "transitionMap": {
    "q0": [
      {
        "label": "0/0R",
        "sourceid": "q0",
        "targetid": "q0"
      },
      {
        "label": "1/1R,B/BR",
        "sourceid": "q0",
        "targetid": "q1"
      }
    ],
    "q1": [
      {
        "label": "0/0R",
        "sourceid": "q1",
        "targetid": "q1"
      },
      {
        "label": "1/1R",
        "sourceid": "q1",
        "targetid": "q0"
      },
      {
        "label": "B/BR",
        "sourceid": "q1",
        "targetid": "q3"
      }
    ],
    "q2": [
      {
        "label": "",
        "sourceid": "q2",
        "targetid": "q3"
      }
    ],
    "q3": [
      {
        "label": "",
        "sourceid": "q3",
        "targetid": "q3"
      }
    ]
  },
  "finalNodes": ["q3"],
  "nodeName": "q3",
  "nodeNum": 4,
  "automataType": "TM",
  "inputString": "111",
  "stageProps": {
    "x": 294.17799999999998,
    "y": 265.80199999999996,
    "scale": 1.3310000000000004,
    "draggable": true
  },
  "stateOutput": {},
  "tapeMode": "single",
  "inputString2": ""
}
```

Result: **Accepted**. Correct answer: **Accepted**. This matches.



Repo files: q6_run5_fixed_empty.json, q6_run5_fixed_empty.png, README line "Q6 run 5"

JSON export (q6_run5_fixed_empty.json):

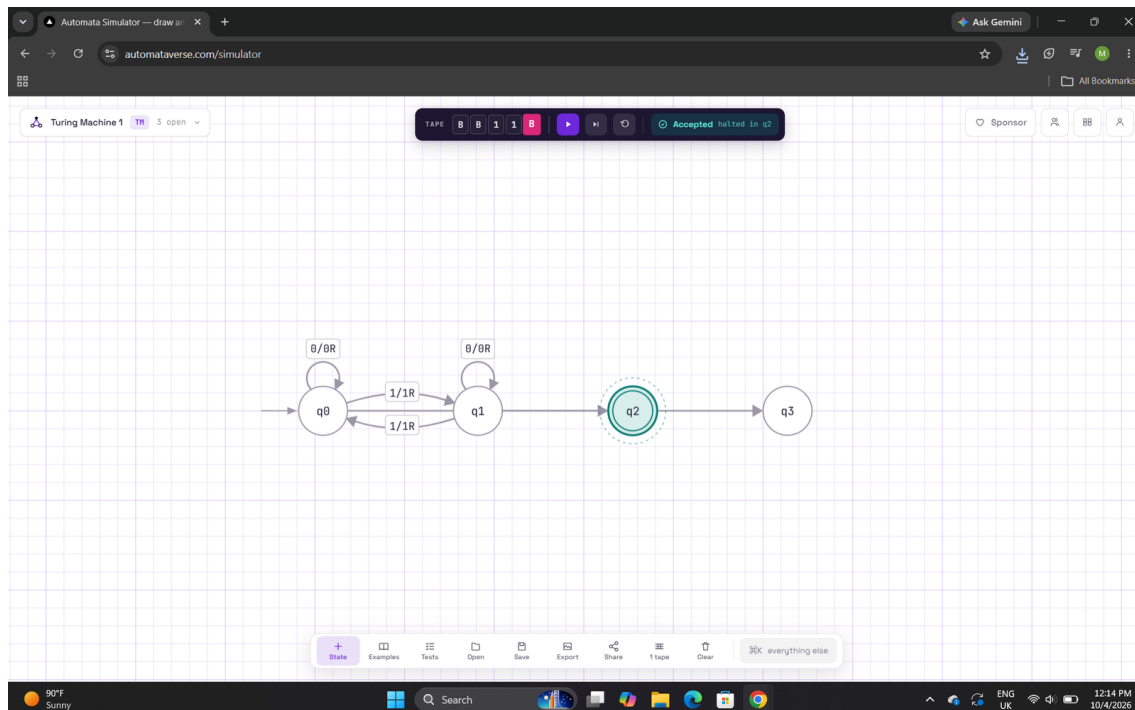
```

{"nodeMap":{"q0":{"id":"q0","x":120,"y":110},"q1":{"id":"q1","x":310,"y":110},"q2":{"id":"q2","x":500,"y":110},"q3":{"id":"q3","x":690,"y":110},"transitionMap":{"q0":{"label":"0/0R","sourceid":"q0","targetid":"q0"},"label":"1/1R","sourceid":"q0","targetid":"q1"},"label":"B/BR","sourceid":"q0","targetid":"q2"},"q1":{"label":"0/0R","sourceid":"q1","targetid":"q1"},"label":"1/1R","sourceid":"q1","targetid":"q0"},"label":"B/BR","sourceid":"q1","targetid":"q3"}},
"finalNodes":["q2"],"nodeNameType":"TM","inputString":"","stageProps":{"x":293.3683760198439,"y":303.59083763538564,"scale":1.0999999999999999,"draggable":true},"stateOutput":{"tapeMode":"single","inputString2":""}}

```

Run 6: corrected machine on "11"

Result: **Accepted**. Correct answer: **Accepted**. This matches.



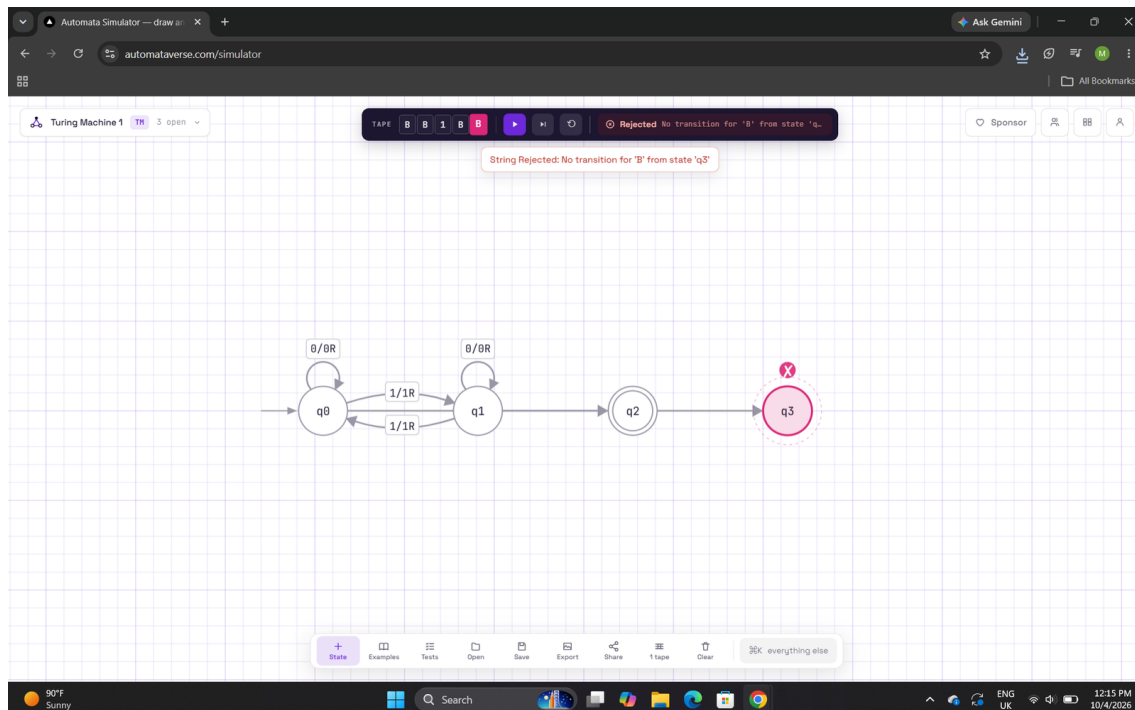
Repo files: q6_run6_fixed_11.json, q6_run6_fixed_11.png, README line "Q6 run 6"

JSON export (q6_run6_fixed_11.json):

```
{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 120,
      "y": 110
    },
    "q1": {
      "id": "q1",
      "x": 310,
      "y": 110
    },
    "q2": {
      "id": "q2",
      "x": 500,
      "y": 110
    },
    "q3": {
      "id": "q3",
      "x": 690,
      "y": 110
    }
  },
  "transitionMap": {
    "q0": [
      {
        "label": "0/0R",
        "sourceid": "q0",
        "targetid": "q0"
      },
      {
        "label": "1/1R",
        "sourceid": "q0",
        "targetid": "q1"
      },
      {
        "label": "B/BR",
        "sourceid": "q0",
        "targetid": "q2"
      }
    ],
    "q1": [
      {
        "label": "0/0R",
        "sourceid": "q1",
        "targetid": "q1"
      },
      {
        "label": "1/1R",
        "sourceid": "q1",
        "targetid": "q0"
      },
      {
        "label": "B/BR",
        "sourceid": "q1",
        "targetid": "q3"
      }
    ],
    "q2": [
      {
        "label": "B/BR",
        "sourceid": "q2",
        "targetid": "q3"
      }
    ]
  },
  "finalNodes": [
    "q2"
  ],
  "nodeNum": 4,
  "automataType": "TM",
  "inputString": "11",
  "stageProps": {
    "x": 293.3683760198439,
    "y": 303.59083763538564,
    "scale": 1.0999999999999999,
    "draggable": true,
    "stateOutput": {},
    "tapeMode": "single",
    "inputString2": ""
  }
}
```

Run 7: corrected machine on "1"

Result: **Rejected**. Correct answer: **Rejected**. This matches.



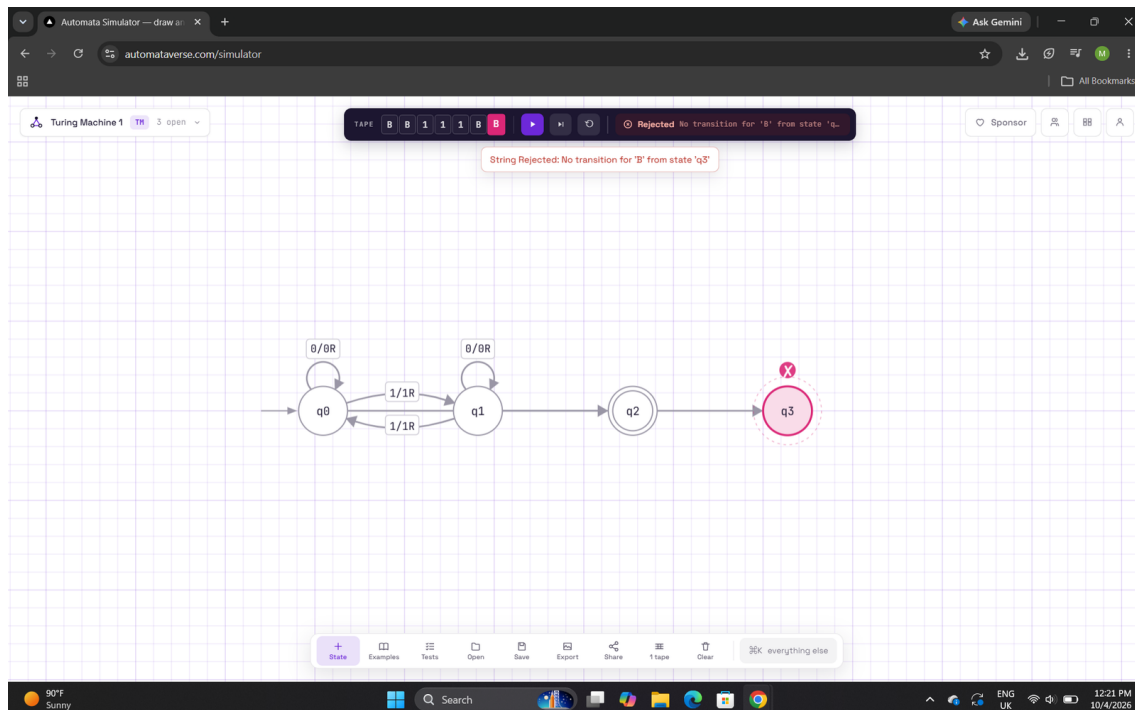
Repo files: q6_run7_fixed_1.json, q6_run7_fixed_1.png, README line "Q6 run 7"

JSON export (q6_run7_fixed_1.json):

```
{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 120,
      "y": 110
    },
    "q1": {
      "id": "q1",
      "x": 310,
      "y": 110
    },
    "q2": {
      "id": "q2",
      "x": 500,
      "y": 110
    },
    "q3": {
      "id": "q3",
      "x": 690,
      "y": 110
    }
  },
  "transitionMap": {
    "q0": [
      {
        "label": "0/0R",
        "sourceid": "q0",
        "targetid": "q0"
      },
      {
        "label": "1/1R",
        "sourceid": "q0",
        "targetid": "q1"
      },
      {
        "label": "B/BR",
        "sourceid": "q0",
        "targetid": "q2"
      }
    ],
    "q1": [
      {
        "label": "0/0R",
        "sourceid": "q1",
        "targetid": "q1"
      },
      {
        "label": "1/1R",
        "sourceid": "q1",
        "targetid": "q0"
      },
      {
        "label": "B/BR",
        "sourceid": "q1",
        "targetid": "q3"
      }
    ],
    "q2": [
      {
        "label": "B/BR",
        "sourceid": "q2",
        "targetid": "q3"
      }
    ]
  },
  "finalNodes": [
    "q2"
  ],
  "nodeNum": 4,
  "automataType": "TM",
  "inputString": "1",
  "stageProps": {
    "x": 293.3683760198439,
    "y": 303.59083763538564,
    "scale": 1.0999999999999999,
    "draggable": true
  },
  "stateOutput": {},
  "tapeMode": "single",
  "inputString2": ""
}
```

Run 8: corrected machine on "111"

Result: **Rejected**. Correct answer: **Rejected**. This matches.



Repo files: q6_run8_fixed_111.json, q6_run8_fixed_111.png, README line "Q6 run 8"

JSON export (q6_run8_fixed_111.json):

```
{
  "nodeMap": {
    "q0": {
      "id": "q0",
      "x": 120,
      "y": 110
    },
    "q1": {
      "id": "q1",
      "x": 310,
      "y": 110
    },
    "q2": {
      "id": "q2",
      "x": 500,
      "y": 110
    },
    "q3": {
      "id": "q3",
      "x": 690,
      "y": 110
    }
  },
  "transitionMap": {
    "q0": [
      {
        "label": "0/0R",
        "sourceid": "q0",
        "targetid": "q0"
      },
      {
        "label": "1/1R",
        "sourceid": "q0",
        "targetid": "q1"
      }
    ],
    "q1": [
      {
        "label": "B/BR",
        "sourceid": "q1",
        "targetid": "q2"
      },
      {
        "label": "1/1R",
        "sourceid": "q1",
        "targetid": "q0"
      }
    ],
    "q2": [
      {
        "label": "0/0R",
        "sourceid": "q2",
        "targetid": "q3"
      }
    ]
  },
  "finalNodes": [
    "q3"
  ],
  "nodeNum": 4,
  "automataType": "TM",
  "inputString": "111",
  "stageProps": {
    "x": 293.3683760198439,
    "y": 303.59083763538564,
    "scale": 1.0999999999999999,
    "draggable": true
  },
  "stateOutput": {},
  "tapeMode": "single",
  "inputString2": ""
}
```

Summary: broken vs corrected vs correct answer

Input	Broken machine	Corrected machine	Correct answer
""	Rejected	Accepted	Accepted
"11"	Rejected	Accepted	Accepted
"1"	Rejected	Rejected	Rejected
"111"	Rejected	Rejected	Rejected

The broken machine is wrong on "" and "11" and right on "1" and "111". The corrected machine is right on all four.

Task D - Reflection

1 and 111 wouldn't have shown the bug. They both have an odd number of 1s so they should be rejected, and the broken machine rejects everything, so it got them right by luck. Only "" and 11 should be accepted, and those are the ones that show the problem. When I test a machine I should use strings that should be accepted and strings that should be rejected, and edge cases like the empty string. I should also check that the accept state can actually be reached.